

Java Coffee — The Barista's Quest

An Object-Oriented Console RPG with Player-Authored Game Rules



Course: Object-Oriented Programming (OOP)

Curriculum: Data Analysis

Dipartimento di Scienze Matematiche e Informatiche, Scienze Fisiche e Scienze
della Terra

Bachelor's Degree in Computer Science

Academic Year 2025/2026

Università degli Studi di Messina

Supervisor:

Prof. Salvatore Distefano

Student:

Nasywa Ramadhani (574106)

Submission Date: July 2nd, 2026

Abstract

This report presents Java Coffee — The Barista's Quest, a console-based card-battle RPG built in Java SE 17 as my OOP final project at the University of Messina. The game demonstrates all required OOP concepts — abstraction, encapsulation, inheritance, polymorphism, composition, modularity, and exception handling — through a turn-based battle system where a barista fights enemies across three Indonesian coffee regions.

The main feature I am most proud of is the **player-authored rule system**. Before each game, the player can type IF/THEN rules in plain text — for example, `IF player.health < 30 THEN player.gain("OatMilk")`. Each rule is parsed into a real Java object and evaluated automatically every turn during battle. Enemies can also create their own rules mid-battle — for example, HealthInspector bans ATTACK cards for 2 turns. This demonstrates that rules are first-class objects that any entity can create at runtime.

Table of Contents

<u>Rebuttal to Supervisor Remarks</u>	4
<u>1. Introduction & Project Scope</u>	7
<u>1.1 Project Overview</u>	7
<u>1.2 Motivation and Objectives</u>	7
<u>1.3 Development Environment and Tools</u>	7
<u>2. System Architecture</u>	8
<u>2.1 Game Setting and Theme</u>	8
<u>2.2 Package Organization</u>	8
<u>2.3 Core Gameplay Loop</u>	9
<u>2.4 Customization Features and Simulation Complexity</u>	9
<u>2.5 Starter Deck — Card Reference</u>	11
<u>3. OOP Concepts Demonstrated</u>	11
<u>3.1 Abstraction</u>	11
<u>3.2 Encapsulation</u>	12
<u>3.3 Information Hiding</u>	14
<u>3.4 Inheritance</u>	16
<u>3.5 Subtyping</u>	17
<u>3.6 Interfaces (Multi-typing)</u>	18
<u>3.6.1 Multi-typing Demonstrated from the Client Program</u>	19
<u>3.7 Polymorphism</u>	20
<u>3.7.1 Subtype Polymorphism (Inclusion)</u>	20
<u>3.7.2 Parametric Polymorphism (Generics)</u>	22
<u>3.7.3 Ad-hoc Polymorphism (Overloading)</u>	23
<u>3.7.4 Coercion Polymorphism (Type Casting / Interface Widening)</u>	24
<u>3.8 Modularity</u>	25
<u>3.9 Composition</u>	28
<u>3.10 Hierarchy</u>	29
<u>3.11 Reuse</u>	32
<u>3.12 Extensibility</u>	34
<u>3.13 Exception Handling</u>	35

<u>4. Player-Authored Game Rules</u>	38
<u>4.1 The GameRule Interface</u>	38
<u>4.2 Concrete Rule Implementations</u>	39
<u>4.3 The IF/THEN RuleParser</u>	41
<u>4.4 Dynamic Rule Mutation by Enemies</u>	42
<u>5. UML Diagrams</u>	44
<u>5.1 Model Package</u>	44
<u>5.2 Battle Package</u>	45
<u>5.3 Items and World Packages</u>	45
<u>6. Game Logic and World Extensions</u>	47
<u>6.1 Status Effects</u>	47
<u>6.2 Procedural Encounters</u>	50
<u>6.3 Difficulty Scaling</u>	51
<u>6.4 Equipment Bonus System</u>	53
<u>6.5 Brewmaster Boss — Two-Phase Battle</u>	54
<u>6.6 RunSummary and Score Log</u>	55
<u>7. Sample Run Transcript</u>	56
<u>7.1 Setup — Configuring Rules and Authoring a GameRule</u>	57
<u>7.2 Battle — A Player-Authored Rule Fires</u>	57
<u>7.3 Battle — The Player Silences an Enemy (BanCard)</u>	58
<u>8. Technical Challenges and Design Decisions</u>	58
<u>8.1 Single Static Scanner — Preventing Input Buffering Bugs</u>	58
<u>8.2 While Loop Instead of Recursion for Re-prompt</u>	59
<u>8.3 Defensive Copy in the Rule Evaluation Loop</u>	60
<u>8.4 instanceof in BanCard and AttackCard</u>	61
<u>8.5 Hand Management — Clearing Between Battles</u>	62
<u>8.6 Unifying Player Rules and Enemy Card Bans</u>	63
<u>9. Extensibility and Future Enhancements</u>	65
<u>10. Reflection and Learning Outcomes</u>	66
<u>11. Conclusion</u>	67

Rebuttal to Supervisor Remarks

This section provides a **detailed point-by-point** response to the remarks raised by Prof. Distefano on the previous version of the report and implementation. For each remark, the corresponding modifications made to both the source code and the report are described so that every correction can be verified directly.

Remark 1

"Encapsulation and inf hiding are two different concepts, now they are overlapped and mixed up in encapsulation, some concepts of the latter are used in the former generating confusion."

Response — Accepted and fixed.

Sections **3.2 (Encapsulation)** and **3.3 (Information Hiding)** have been completely separated. Section 3.2 now discusses only the bundling of data and behaviour into a class, while all concepts related to visibility, access control, and hidden implementation details have been moved to Section 3.3. This eliminates the overlap and aligns the discussion with the OOP course material.

Remark 2

"Inf hiding is not well enforced. There are non-private attributes despite your claims. Inf hiding should enforce the 3-step inf hiding engineering (private, class or instance, constant or variable)."

Response — Accepted and fixed.

The remark identified two distinct failures: a violation of Step 1, and the absence of Steps 2 and 3 as an explicit engineering process. Both have been addressed in the code, and Section 3.3 has been rewritten around the three steps rather than around access modifiers alone.

Step 1 — Access level (private). I audited all 61 classes. One attribute was not private: `public int damage` in `battle/AttackCard.java`. It is now `private final int damage`, exposed through a read-only accessor `public int getDamage()`. The two external classes that previously read the field directly were updated to use the accessor: `ComboCard.describe()` (two accesses) and `BattleEngine.playerTurn()` (one access, in the RunSummary damage bookkeeping) — two classes, three call sites. After this change **every one of the 116 attributes in the project is private**; no field is readable or writable from outside its declaring class.

Step 2 — Class member or instance member. `static` is now applied only where a value belongs to the class as a whole rather than to any single object. There are exactly 15 such attributes:

- **SC : Scanner** — one per class that reads console input (**Main**, **BattleEngine**, **ShopEncounter**). A single shared input resource per class; see Section 8.1.
- **RANDOM : Random** — one per enemy class that selects a move at random (**AngryCustomer**, **RivalBarista**, **HealthInspector**, **CoffeeCritic**, **Brewmaster**). One generator per enemy type, not one per spawned enemy.
- Seven compiled **Pattern** constants in **RuleParser** (**HEALTH_GAIN**, **HEALTH_HEAL**, **STAMINA_DRAW**, **TURN_DRAW**, **LEGACY_STAMINA**, **EVERY_DRAW**, **COMBO_BOOST**). The grammar is a property of the parser, not of a parse.

Every other attribute — all 101 of them — is an instance member, because it describes the state of one particular object (this barista's health, this card's damage, this rule's threshold).

Step 3 — Constant or variable. **final** is now applied to every attribute that is assigned once and never reassigned afterwards. This was enforced mechanically: for each candidate field I added **final** and recompiled; the compiler rejects **final** on any field that is reassigned, so the resulting 68 instance constants and 15 class constants are *proved* immutable rather than assumed to be.

Exactly 33 attributes remain variable, and each is genuinely mutable state that changes during play: **health**, **stamina**, **maxHealth**, **maxStamina** (Character); **beans**, **comboBonus** (Barista); **silencedTurns** (Enemy); **phase** (Brewmaster); **turnsRemaining**, **justApplied** (StatusEffect); **fumbled** (Jittery); the **triggered** / **applied** / **hasInspected** / **hasReviewed** one-shot flags; **equipped** (Equipment); the seven RunSummary counters; the five RuleSet configuration values; and **cardsPlayedThisTurn** (BattleContext). Each of these changes only through a validating method — **takeDamage()** clamps at zero, **heal()** clamps at **maxHealth**, **useStamina()** throws **InvalidBrewException** rather than going negative, and **setComboBonus()** clamps at zero.

Notably, **no attribute in the project is simultaneously a class member and a variable**: there is no mutable global state anywhere in the codebase.

Report Fix. Section 3.3 (Information Hiding) now presents the three steps as a decision procedure applied to every attribute, with Table IH-1 giving a per-attribute analysis covering all three categories that occur (private+instance+variable, private+instance+constant, private+class+constant) and noting the absence of the fourth. Sections 3.2 and 3.3 have also been fully separated per Remark 1. [See Section 3.3, Table IH-1.](#)

Remark 3

"Subtyping–multityping should be shown by an example from the client program where you demonstrate a multitype object can be accessed by different types of variables."

Response — Accepted and fixed.

A client-side demonstration has been added to `Main.java` through the `demonstrateMultityping()` method. The report now includes the new subsection [3.6.1 Multi-typing Demonstrated from the Client Program](#), showing the same object accessed through different reference types and explaining how interface-based multi-typing works in practice.

Remark 4

"Customization feature section is missing."

Response — Accepted and fixed.

Section 2.4 has been renamed “[Customization Features and Simulation Complexity](#)” and lists everything the player can customize: game settings, barista type, equipment, and the player-authored IF/THEN rule system, with a cross-reference to Chapter 4 where the rule system is discussed in detail.

1. Introduction & Project Scope

1.1 Project Overview

Java Coffee — The Barista's Quest is a console-based RPG in which the player assumes the role of a barista on a quest across three Indonesian coffee regions — Warung Jogja, Solo Heritage, and Dieng Highlands — battling rival brewers in card combat, collecting rare beans, and aiming for the title of Grand Master Barista.

I built this game as my OOP final project, satisfying the requirement for a console-only simulation with rich game logic. Every game mechanic is a direct expression of an OOP concept: a Barista fights an Enemy (inheritance + polymorphism), a CoffeeDeck holds CoffeeCards (composition + generics), an Inventory holds Items (parametric polymorphism), and a RuleSet controls what is allowed during battle (extensibility + dynamic dispatch).

1.2 Motivation and Objectives

This project came from two motivations — one technical, one cultural.

On the technical side, Prof. Distefano's feedback on my proposal made it clear that rule customization should mean the player actually creates new rules, not just picks from a list. That's what pushed me to build the IF/THEN rule system where the player types rules in plain text and the program turns them into real Java objects at runtime.

On the cultural side, Indonesia is one of the world's biggest coffee producers, and the island of Java gave its name to Java coffee — the same coffee that inspired James Gosling to name the programming language in 1995. Building a coffee-themed game in Java felt like a natural fit.

My four objectives for this project were:

- Implement all OOP concepts in a meaningful way, not just as a textbook example
- Deliver a fully playable console game
- Make player-authored rules the main customization feature
- Document every design decision with real code from the actual implementation

1.3 Development Environment and Tools

- Language: Java SE 17
- IDE: NetBeans 19
- Build Tool: Apache Ant (build.xml)
- Libraries: Java Standard Library only — no external dependencies

- Version Control: GitHub (github.com/nasywaarr/barista-quest)
- Platform: Windows 10/11, console-based (no GUI)

2. System Architecture

2.1 Game Setting and Theme

The game is set across three regions, each inspired by real Indonesian coffee culture:

- Warung Jogja — Easy starting region, modelled on Yogyakarta's street-side kopi warung culture. One battle (Angry Customer), one random event, and one shop.
- Solo Heritage — Medium region, inspired by Surakarta's refined, colonial-era coffee houses. One battle (Solo Rival), one random event, and one shop.
- Dieng Highlands — Hard final region, a misty volcanic plateau. Three battles (Health Inspector → Coffee Critic → Brewmaster boss), one event, and one shop.

Each region builds its own encounter pool and shuffles it at the start of the game, except Dieng Highlands where the Brewmaster always appears last. All encounters — BattleEncounter, ShopEncounter, and RandomEvent — are resolved polymorphically by WorldManager without any type-checking.

2.2 Package Organization

I organised the codebase into five packages, each with one clear responsibility:

- model — Character hierarchy (Barista, SpecialistBarista, Enemy and its five subtypes), StatusEffect hierarchy (Burned, Steamed, Jittery), Brewable and Describable interfaces.
- battle — CoffeeCard hierarchy (AttackCard, HealCard, DebuffCard, ComboCard, BanCard), CoffeeDeck<T>, BattleEngine, RuleSet, RunSummary, and the entire GameRule system.
- items — Item hierarchy (Ingredient, Equipment, Collectible) and the generic Inventory<T> container.
- world — Region hierarchy (WarungJogja, SoloHeritage, DiengHighlands), Encounter hierarchy (BattleEncounter, ShopEncounter, RandomEvent), and WorldManager.
- exceptions — Five custom checked exceptions: EmptyDeckException, InvalidBrewException, OutOfStockException, BattleOverException, and InvalidRuleException.

2.3 Core Gameplay Loop

Every battle follows the same loop, which I implemented in `BattleEngine.runBattle()`:

1. Reset the hand and stamina, shuffle the deck, and draw the opening hand.
2. At the start of each turn, build a `BattleContext` snapshot and check every `GameRule` — any rule whose `appliesTo()` returns true has its `execute()` called immediately.
3. Player turn — display the hand with stamina and ban indicators, prompt the player to choose a card, then play it.
4. Tick all status effects (Burned, Steamed, Jittery) on both the player and the enemy.
5. Enemy turn — call `enemy.act(barista)`, which is a final method in `Enemy` that delegates to the protected `decideAction()`. If the enemy is Jittery and fumbles, their turn is skipped.
6. Regenerate stamina (+1 for both). Decrement the card ban duration in `RuleSet`.

This loop runs identically regardless of which concrete `Enemy` subtype or which `CoffeeCard` subtype is involved — Section 3.7 explains exactly how polymorphism makes this possible.

2.4 Customization Features and Simulation Complexity

Customization is a first-class feature of Java Coffee, offered at three levels:

- (1) Configuration customization: at game start the player sets deck size (5–12), hand size (2–10), max stamina (5–20), difficulty (1–3), and permadeath, and chooses a barista specialization (Standard, Ice (+2 max stamina), Espresso (+10 max HP)).
- (2) Run customization: equipment such as the `MokaPot` purchased from shop grants +5 permanent attack bonus for the rest of the run.
- (3) Rule customization — the core of the project: the player does not pick rules from a menu but authors brand-new game rules from scratch, typing free-text IF/THEN statements that are parsed into live `GameRule` objects evaluated every turn; enemies use the same mechanism to inject their own rules mid-battle. The design and implementation of this non-parametric rule customization is developed in full in Section 4 (Player-Authored Game Rules).

Since Java Coffee has no graphical interface, I focused on making the simulation logic as rich and complex as possible. Here is what the system contains:

Entities and Class Hierarchy (61 Java files)

- Characters: `Barista`, `SpecialistBarista` (ICE, ESPRESSO), and 5 enemy types (`AngryCustomer`, `RivalBarista`, `HealthInspector`, `CoffeeCritic`, `Brewmaster`).

- Cards: 5 CoffeeCard subtypes (AttackCard, HealCard, DebuffCard, ComboCard, BanCard) with 7 unique cards in the starter deck.
- Items: Ingredient (OatMilk, EspressoShot, SimpleSyrup), Equipment (MokaPot — actively affects combat), Collectible (RareBean).
- Status Effects: Burned (damage per turn), Steamed (damage + stamina drain), Jittery (50% fumble chance).
- Regions and Encounters: 3 regions, 3 encounter types (BattleEncounter, ShopEncounter, RandomEvent), procedurally shuffled per run.
- Rule System: 7 concrete GameRule classes — 6 player-authored at runtime via IF/THEN grammar, plus 1 enemy-only rule (**BannedCardRule**).

Customisable Rules (Non-Parametric)

Instead of picking a rule from a menu, the player writes rules from scratch:

```
--- Custom Rules ---
Write your own rules. They fire automatically every turn.
Separate multiple rules with ;

IF player.health < 30 THEN player.heal(15)
IF player.health < 30 THEN player.gain(X)
IF player.stamina < 3 THEN draw(2)
IF turn % 3 == 0 THEN draw(1)
COMBO 2 THEN damage_boost_20

X = OatMilk (+10 HP) EspressoShot (+10 ST) SimpleSyrup (+5 HP +5 ST)

Tip: IF player.health < 30 THEN player.gain("OatMilk"); IF turn % 3 == 0 THEN draw(1)

(Press Enter to skip)
Your rules: IF player.health < 30 THEN player.gain("OatMilk"); IF turn % 3 == 0 THEN draw(1)
OK: IF player.health < 30 THEN player.gain("OatMilk")
OK: Periodic Draw: Every 3 turns, gain 1 card(s)
2 rule(s) active this run.
```

Each line becomes a brand-new Java object that did not exist before the player typed it. Enemies can also inject rules mid-battle — HealthInspector bans ATTACK cards, CoffeeCritic bans HEAL cards.

State Tracked Per Run

- Per character: health, stamina, active status effects, beans, equipment bonus.
- Per battle: turn count, card ban type and duration, rule trigger states.
- Per run: total turns, cards played, damage dealt, healing done, enemies defeated, score, and a battle log.

Rich Console Output

Since there is no GUI, every game state change is printed clearly to the console — card plays with damage numbers, status effect ticks, rule triggers, card bans, enemy moves, shop transactions, and a full RunSummary at the end.

2.5 Starter Deck — Card Reference

The default deck starts with these 7 unique cards, covering four of the five CoffeeCard subtypes (ATTACK, HEAL, DEBUFF, COMBO). The fifth subtype, BanCard, enters play as *Ristretto Lock* when the player sets a deck size above 7, drawn from a pool of extra cards:

Card	Type	Cost	Effect
Espresso Shot	ATTACK	2 ST	15 damage
Americano	ATTACK	3 ST	22 damage
Cafe Crème	ATTACK	1 ST	8 damage
Warm Milk	HEAL	0 ST	+4 stamina (free)
Oat Latte	HEAL	1 ST	+20 HP
Cappuccino	DEBUFF	2 ST	Steamed (3 dmg + ST drain x2 turns)
Double Shot Combo	COMBO	4 ST	15+15 = 30 damage

3. OOP Concepts Demonstrated

This section explains how each core OOP concept is realised in Java Coffee, with a definition, motivation, how I applied it, a code example from the actual implementation, and my commentary on the design choice.

All code examples are taken directly from the github.com/nasywaarr/barista-quest repository.

3.1 Abstraction

Definition: Abstraction means defining a contract — what an object must do — without specifying how. In Java this is done through abstract classes (which can provide shared concrete methods) and interfaces (which provide none).

Why it matters: BattleEngine can run a battle between a Barista and any Enemy subtype — including ones that don't exist yet — because it only ever calls methods declared on the abstract supertype. The engine is never coupled to any concrete class.

How I applied it: I defined six abstract classes — Character, CoffeeCard, Enemy, Region, Encounter, and StatusEffect — each acting as a contract that subclasses must fulfil. For example, every Character must implement `act()`, but each subclass decides what that action does.

Code example — model/Character.java:

```
public abstract class Character {
    private String name;
    private int health;
    private int maxHealth;
    private int stamina;
    private int maxStamina;
    private List<StatusEffect> statusEffects;

    public abstract void act(Character target) throws BattleOverException;

    public void takeDamage(int amount) throws BattleOverException {
        if (amount < 0) amount = 0;
        health = Math.max(0, health - amount);
        if (health == 0)
            throw new BattleOverException(name + " has been defeated!");
    }
}
```

Listing 1: Character.java — abstract `act()` alongside concrete `takeDamage()`

Commentary: Character defines what all game entities must do without deciding how each one does it. Barista and RivalBarista behave completely differently when they act, but both inherit the same `takeDamage()` without writing it twice. This is the core benefit of abstraction — shared behaviour in one place, specialised behaviour in each subclass.

3.2 Encapsulation

Definition: Encapsulation is the bundling of data (attributes) and the behaviour that operates on that data (methods) into a single unit — the class. The object is a capsule: state and the operations that manipulate that state live together, travel together, and are defined in one place.

Why it matters: Without encapsulation, the data describing a barista and the logic that manipulates a barista would be scattered across the program, and every piece of code would need to know how the others represent that data. With encapsulation, everything that defines what a Character is and does lives in `Character.java`: its state (name, health, stamina, status effects) and its operations (`takeDamage()`, `heal()`, `useStamina()`, `tickStatusEffects()`) form one cohesive unit.

How I applied it: Every concept of the game is modelled as one class bundling its state with its behaviour. Character bundles vital statistics with the methods that alter them. `CoffeeDeck<T>` bundles the three card piles (draw, hand, discard) with the operations that move cards between them (`recycleDiscard()`). `Inventory<T>` bundles the item list with `add()/remove()`. `RuleSet` bundles the rule list and the ban state with `addRule()`, `banCardType()`, and `isCardAllowed()`. In every case, the data and the only logic entitled to operate on that data are defined in the same unit.

Code example — model/Character.java:

```
public abstract class Character {
    private String name;
    private int health;
    private int maxHealth;
    private int stamina;
    private int maxStamina;
    private List<StatusEffect> statusEffects;

    public void useStamina(int cost) throws InvalidBrewException {
        if (cost > stamina)
            throw new InvalidBrewException(
                name + " needs " + cost + " stamina but only has " + stamina + "!");
        stamina -= cost;
    }
}
```

Code example — battle/CoffeeDeck.java:

```
public class CoffeeDeck<T extends CoffeeCard> {
    private List<T> cards;
    private List<T> hand;
    private List<T> discard;

    public CoffeeDeck() {
        this.cards = new ArrayList<>();
        this.hand = new ArrayList<>();
        this.discard = new ArrayList<>();
    }

    public void discard(T card) {
        hand.remove(card);
        discard.add(card);
    }
}
```

Listing 2: Encapsulated state mutation in Character and CoffeeDeck

Commentary: The clearest benefit appeared during debugging, when stamina behaved incorrectly, there was exactly one place to look — `Character` — because stamina and every operation touching it are encapsulated there. How access to that bundled state is restricted is a different concern, treated separately in Section 3.3 (Information Hiding).

3.3 Information Hiding

Definition: Information hiding is the concealment of a class's internal representation and decision-making behind a stable public interface. Clients can only use the class through that interface — they can't read, modify, or depend on its internals. Encapsulation puts data and behaviour together (Section 3.2); information hiding decides what of that bundle is actually visible from outside.

The 3-step information hiding engineering. Following the course slides, I applied three decisions to every attribute in the project:

Step 1 — Access level: Every attribute is **private**. No exceptions anywhere in the codebase, including inside the class hierarchies: subclasses reach superclass state through **protected** accessors (**setMaxHealth()**, **setMaxStamina()**), never through direct field access.

Step 2 — Class member or instance member: **static** is used only when a value belongs to the class as a whole: the shared **Scanner** in each input-reading class, the **Random** generator in each enemy class, and **RuleParser**'s seven compiled **Pattern** constants. All 101 remaining attributes are instance members.

Step 3 — Constant or variable: **final** is used wherever a value is assigned once and never reassigned. This was verified by the compiler rather than by inspection: **final** was added to every candidate and the project recompiled, so the 83 constants are provably immutable. The 33 variables are exactly the attributes that must change during play, and each changes only through a validating method.

Table IH-1 applies these three steps to representative attributes:

Attribute (class)	Step 1 — Access	Step 2 — Class / Instance	Step 3 — Constant / Variable	Why
health, maxHealth, stamina, maxStamina (Character)	private	instance	variable	Per-entity mutable state; changed only by takeDamage(), heal(), useStamina(), restoreStamina(), all of which clamp or throw
name (Character)	private	instance	constant (final)	Identity is fixed at construction
statusEffects (Character)	private	instance	constant (final)	The list reference never changes; contents change through addStatusEffect() / tickStatusEffects(); exposed via unmodifiableList
damage, drawOnPlay (AttackCard)	private	instance	constant (final)	A card's damage is fixed when the card is created; read-only accessor getDamage(), so it can be read but never rewritten mid-battle
attackPower, reward (Enemy)	private	instance	constant (final)	Set once from the difficulty formula in the constructor
silencedTurns (Enemy)	private	instance	variable	Counts down inside the final act(); invisible to BattleEngine
specialization (SpecialistBarista)	private	instance	constant (final)	Chosen once at construction; hidden behind applyBonus()
cards, hand, discard (CoffeeDeck<T>)	private	instance	constant (final)	The three pile references never change; the pile invariant (a card is never in two piles) is protected by draw() / discard() / recycleDiscard()
allowedCardTypes, bannedCardDurations, customRules (RuleSet)	private	instance	constant (final)	Container references fixed at construction; mutated only via banCardType(), tick(), addRule()
startingDeckSize, handSize, maxStamina, enemyScalingFactor, permadeath (RuleSet)	private	instance	variable	Set by configure(), which validates every range and throws InvalidRuleException
healthThreshold, itemName (ItemGainRule)	private	instance	constant (final)	The rule the player authored cannot change after parsing
triggered (ItemGainRule, HealRule, StaminaDrawRule, ConditionalBonusRule)	private	instance	variable	The one-shot latch: flipped in execute(), tested in appliesTo()
ruleSet, bannedType, turns (BannedCardRule)	private	instance	constant (final)	Fixed when the enemy injects the rule
applied (BannedCardRule)	private	instance	variable	One-shot latch, as above
totalDamageDealt, score, ... (RunSummary)	private	instance	variable	Accumulators; incremented only through addDamage(), addScore(), ...
SC : Scanner (Main, BattleEngine, ShopEncounter)	private	class (static)	constant (final)	One shared input resource per class, never reassigned — see Section 8.1
RANDOM : Random (the five Enemy subclasses)	private	class (static)	constant (final)	One generator per enemy type, not per spawned enemy
HEALTH_GAIN ... COMBO_BOOST : Pattern (RuleParser)	private	class (static)	constant (final)	The IF/THEN grammar is a property of the parser; seven immutable compiled patterns shared by all parses

Table IH-1: The 3-step information hiding engineering (access · class/instance · constant/variable) applied across the codebase.

Enforcement in code. `AttackCard.damage` used to be `public`, read directly by `ComboCard` and `BattleEngine`. It is now `private final`, exposed only through the read-only `getDamage()`. Step 1 holds with no exceptions, and Step 3 is enforced by the compiler rather than by convention.

The same principle covers the getters. `Character.getStatusEffects()`, `Inventory.getAll()`, `CoffeeDeck.getHand()`, and `RuleSet.getRules()` all return `Collections.unmodifiableList(...)` instead of the raw field. Clients can read the contents but cannot add, remove, or reorder them — changes happen only through the owning class's own methods. Returning the live list would have handed clients a private field through a public door.

Why it matters: Information hiding covers decisions, not just data. If `BattleEngine` could call an `Enemy`'s AI directly, it would need a special case per enemy type, with nothing stopping it from firing the same action twice in one turn.

How I applied it: `Enemy` declares `act()` as final and delegates to a protected abstract `decideAction()`. Subclasses — `RivalBarista`, `AngryCustomer`, `HealthInspector`, `CoffeeCritic`, `Brewmaster` — override only `decideAction()`; `BattleEngine` only calls `act()`. protected is the right level here: subclasses need to see `decideAction()` to implement it, but clients don't.

Code example — model/Enemy.java:

```
public abstract class Enemy extends Character {
    private int attackPower;
    private int reward;
    private int silencedTurns = 0;

    @Override
    public final void act(Character target) throws BattleOverException {
        if (silencedTurns > 0) {
            System.out.println(" " + getName() + " is silenced and cannot act!");
            silencedTurns--;
            return;
        }
        decideAction(target);
    }

    public void silence(int turns) { this.silencedTurns = turns; }
    public boolean isSilenced() { return silencedTurns > 0; }

    protected abstract void decideAction(Character target) throws BattleOverException;

    public int getAttackPower() { return attackPower; }
    public int getReward() { return reward; }
}
```

Listing 3: Enemy.java — `act()` is final and hides `decideAction()` behind it

Commentary: This keeps `HealthInspector` and `CoffeeCritic` safe — each tracks a private flag (`hasInspected/hasReviewed`) so its special move fires only once, and `BattleEngine` can never re-trigger it since `decideAction()` is protected. `BanCard`'s silence mechanic works the same way: `enemy.silence(1)` sets a private counter inside `act()` that `BattleEngine` never sees. From the engine's side, every enemy is just a `Character` whose `act()` runs once per turn.

The payoff: invalid states become unrepresentable from outside. Health only changes through `takeDamage()` / `heal()`, a card never sits in two piles at once, and an attack's damage can be read but never rewritten mid-battle. Encapsulation bundled the data and the logic; information hiding sealed it shut.

3.4 Inheritance

Definition: Inheritance allows a subclass to acquire the fields and methods of a superclass, optionally overriding some to specialise behaviour. It establishes an IS-A relationship between classes.

Why it matters: Shared behaviour is written once in a superclass and automatically available to every subclass. New character types can be added by extending an existing class and overriding only what differs — no duplication needed.

How I applied it: The project has a three-level inheritance hierarchy: `Character` → `Barista` → `SpecialistBarista`. `Enemy` extends `Character` and is itself extended by five concrete enemy types. This means health management, stamina management, and status effect ticking are all written once in `Character` and inherited by every entity in the game.

Code example — model/Character.java:

```
public abstract class Character {
```

Code example — model/Barista.java:

```
public class Barista extends Character {
```

Code example — model/SpecialistBarista.java:

```
public class SpecialistBarista extends Barista {

    public enum Specialization { ICE, ESPRESSO }
    private Specialization specialization;

    public SpecialistBarista(String name, int health, int stamina, Specialization spec) {
        super(name, health, stamina);
        this.specialization = spec;
        applyBonus();
    }

    private void applyBonus() {
        switch (specialization) {
            case ICE:
                setMaxStamina(getMaxStamina() + 2);
                restoreStamina(2);
                System.out.println(getName() + " — Ice Barista: +2 max stamina!");
                break;
            case ESPRESSO:
                setMaxHealth(getMaxHealth() + 10);
                heal(10);
                System.out.println(getName() + " — Espresso Barista: +10 max HP!");
                break;
        }
    }
}
```

Listing 4: SpecialistBarista.java — three-level hierarchy Character → Barista → SpecialistBarista

Commentary: The three-level hierarchy means `takeDamage()`, `heal()`, `useStamina()`, and `tickStatusEffects()` are written once in `Character` and inherited unchanged by `Barista`, `SpecialistBarista`, `RivalBarista`, `HealthInspector`, `Brewmaster`, and every other entity. When I needed to add the Ice and Espresso specializations, I only had to write what was different — the bonus applied at construction — and inherited everything else from `Barista` and `Character` automatically. The protected setters `setMaxHealth()` and `setMaxStamina()` are accessible to subclasses but not to external code, which is exactly the access level `SpecialistBarista` needs.

3.5 Subtyping

Definition: Subtyping states that an object of a subtype must be usable anywhere an object of its supertype is expected, without breaking the correctness of the program.

Why it matters: `BattleEngine.runBattle()` takes a `Barista` and an `Enemy` as parameters, but internally treats both as `Character` references for shared operations — `takeDamage()`, `tickStatusEffects()`, `isAlive()`. If `Brewmaster` violated the `Character` contract — for example, by allowing health to go negative — the battle loop would behave incorrectly without ever being told why.

How I applied it: Every enemy IS-A `Character`. `BattleEngine` calls `barista.tickStatusEffects()` and `enemy.tickStatusEffects()` — the exact same inherited method — on both, without knowing or caring which concrete subtype the enemy is.

Code example — battle/BattleEngine.java:

```
public void runBattle(Barista barista, Enemy enemy)
    throws EmptyDeckException, InvalidBrewException,
           OutOfStockException, BattleOverException, InvalidRuleException {

    barista.tickStatusEffects();
    enemy.tickStatusEffects();

    ruleSet.tick();

    barista.restoreStamina(1);
    enemy.restoreStamina(1);
}
```

Listing 5: BattleEngine.java — Barista and Enemy substituted wherever Character is expected

Commentary: The Brewmaster has a two-phase enrage mechanic internally, but BattleEngine never sees it — from the engine's perspective, Brewmaster is just another Character whose act() sometimes deals more damage. This is substitutability in practice: I can add any new Enemy subtype and BattleEngine will handle it correctly without a single line of change, as long as the new enemy respects the Character contract.

3.6 Interfaces (Multi-typing)

Definition: An interface is a contract specifying what methods a class must implement, without providing any implementation. A class can implement multiple interfaces simultaneously, gaining more than one type at once — this is called multi-typing.

Why it matters: Interfaces decouple code from class hierarchies. Code written against the Brewable interface works with any object that implements it, regardless of where it sits in the inheritance tree. This enables reuse across completely unrelated hierarchies.

How I applied it: I defined two interfaces — Brewable and Describable. CoffeeCard implements both, and so does Item, even though CoffeeCard and Item share no common superclass. This means the hand display in BattleEngine can call `card.describe()` on any card type — AttackCard, HealCard, DebuffCard, ComboCard, or BanCard — through the same Describable reference.

Code example — model/Brewable.java:

```
package model;

public interface Brewable {
    void brew(Character target);
}
```

Code example — model/Describable.java:

```
package model;

public interface Describable {
    String describe();
}
```

Code example — battle/CoffeeCard.java:

```
public abstract class CoffeeCard implements Brewable, Describable {
```

Code example — items/Item.java (first lines only):

```
public abstract class Item implements Brewable {
```

Listing 6: CoffeeCard implements both; Item implements Brewable — two unrelated hierarchies sharing one interface

Commentary: The most interesting thing about this design is that `CoffeeCard` and `Item` have no inheritance relationship at all — one is a card, the other is a shop item — but both implement `Brewable`. This is exactly what interfaces are for: sharing a contract across unrelated types without forcing them into the same hierarchy. If I had tried to achieve the same thing with inheritance, I would have needed a common superclass for cards and items, which would have been a forced and unnatural design.

One thing worth noting: `CoffeeCard.brew()` is empty. That's on purpose. `Brewable.brew()` doesn't declare any exceptions, but a card's real logic in `use()` needs to throw several — low stamina, empty deck, out of stock, battle over, invalid rule. An overridden method can't throw exceptions the interface doesn't allow, so `brew()` simply can't hold that logic. `CoffeeCard` implements `Brewable` just to prove it is-a `Brewable` for the multi-typing demo in 3.6.1, while the real effect lives in `use()`, which `BattleEngine` actually calls. `Item` has no such exceptions to deal with, so its `brew()` holds the real effect directly.

3.6.1 Multi-typing Demonstrated from the Client Program

To demonstrate multi-typing concretely, the client program (`Main.java`, method `demonstrateMultityping()`, executed at game start) creates a single object and accesses it through three different variable types: its class type and both of its interface types.

Code example — Main.java:

```
Main.java ×
private static void demonstrateMultityping(Barista barista) {
    System.out.println("\n--- Multi-typing demonstration ---");

    CoffeeCard card = new HealCard("Oat Latte", 1, 20); // concrete-class view

    Describable d = card; // SAME object, Describable view
    System.out.println(" As Describable -> " + d.describe());

    Brewable b = card; // SAME object, Brewable view
    System.out.println(" As Brewable -> brew() dispatched on the very same object"
        + " (card instanceof Brewable = " + (card instanceof Brewable) + ")");
    b.brew(barista);

    // Unrelated hierarchy: an Item is ALSO Brewable
    Brewable itemView = new items.OatMilk(10);
    System.out.println(" As Brewable (unrelated Item hierarchy):");
    itemView.brew(barista);

    System.out.println(" One object, three access types: "
        + "CoffeeCard, Describable, Brewable.");
    System.out.println("-----");
}
```

Console output of the demonstration:

--- Multi-typing demonstration ---

As Describable -> [HEAL] Oat Latte (cost: 1 stamina, +20 HP)

As Brewable -> brew() dispatched on the very same object (card instanceof Brewable = true)

As Brewable (unrelated Item hierarchy):

Barista uses OatMilk!

One object, three access types: CoffeeCard, Describable, Brewable.

Commentary: The variable `card` (type `CoffeeCard`), the variable `d` (type `Describable`), and the variable `b` (type `Brewable`) all reference the same single object — no copy is made, and `card instanceof Brewable` confirms at runtime that the object carries all of these types simultaneously. Each variable exposes only the contract of its declared type: through `d` only `describe()` is callable, through `b` only `brew()`. Finally, the same `Brewable` variable type also holds an `OatMilk` from the `Item` hierarchy, which shares no superclass with `CoffeeCard` — the interface contract crosses unrelated hierarchies, which is exactly what multi-typing provides and inheritance alone could not.

3.7 Polymorphism

Definition: Polymorphism means "many forms" — the ability of objects to behave differently depending on their actual type, even when referenced through a common supertype. Java implements four kinds of polymorphism, and I used all four in this project.

Why it matters: Polymorphism enables flexible, extensible code. You write code once against a supertype or interface, and at runtime the correct subtype behaviour is invoked automatically. New subtypes can be added without touching existing code.

3.7.1 Subtype Polymorphism (Inclusion)

Definition: A supertype reference can point to any subtype object. When a method is called on that reference, the JVM dispatches to the correct override at runtime — this is called virtual dispatch.

How I applied it: The `GameRule` evaluation loop in `BattleEngine` is the clearest example. The loop holds a `List<GameRule>` and calls `appliesTo()` and `execute()` on each element — without ever checking what concrete type the rule is. This same loop handles all seven concrete rule types in the project — the six the player can author through `RuleParser` (`ConditionalBonusRule`, `ItemGainRule`, `HealRule`, `StaminaDrawRule`, `PeriodicDrawRule`, `ComboRule`), plus `BannedCardRule`, which enemies like `HealthInspector` and `CoffeeCritic` inject mid-battle.

The same pattern shows up a second time in `WorldManager`, which calls `enc.resolve(barista, engine)` on every `Encounter` — `BattleEncounter`, `ShopEncounter`, and `RandomEvent` — without checking which one it has, and a third time in `Enemy.act()`, which dispatches to whichever `decideAction()` override the concrete enemy subtype provides. In all three cases the calling code was written once, against the abstract type, and it stayed correct as I added new rule types, new encounter types, and new enemies — none of them required touching `BattleEngine` or `WorldManager` at all.

Code example — `battle/BattleEngine.java`:

```

private void applyCustomRules(BattleContext context) {
    List<GameRule> rules = ruleSet.getRules();
    for (GameRule rule : rules) {
        if (rule.appliesTo(context)) {
            rule.execute(context);
        }
    }
}

if (!enemyFumbled) {
    try {
        enemy.act(barista);
    } catch (BattleOverException e) {
        System.out.println(" " + e.getMessage());
        break;
    }
}

```

Listing 7: BattleEngine.java — single call site, multiple possible runtime behaviours

The loop never checks `if rule instanceof ConditionalBonusRule` or casts to specific types. It treats every rule identically, and the JVM dispatches to the correct `execute()` implementation at runtime depending on whether the rule is an `ItemGainRule`, `PeriodicDrawRule`, `ComboRule`, or any future rule type.

Commentary: This is the heart of the dynamic rule system. When I add a new rule type, I just create a new class implementing `GameRule` — `BattleEngine` needs zero changes. The same pattern applies to enemy dispatch: `enemy.act(barista)` calls the correct `decideAction()` for `AngryCustomer`, `HealthInspector`, or `Brewmaster` without any type-checking. This is the Open/Closed Principle in action.

3.7.2 Parametric Polymorphism (Generics)

Definition: Generics allow a single class to work correctly with any compatible type parameter. The same code is reused for different types, with compile-time type safety enforced by the compiler.

Why it matters: Without generics, I would need a separate deck class for each card type and a separate inventory class for each item type — duplicating the same logic many times. Generics let me write the logic once and use it safely for any type.

How I applied it: `CoffeeDeck<T extends CoffeeCard>` is the card container — it works for any card type while guaranteeing at compile time that only `CoffeeCard` subtypes can be added. `Inventory<T extends Item>` works the same way for items.

Code example — battle/CoffeeDeck.java:

```

public class CoffeeDeck<T extends CoffeeCard> {
    private List<T> cards;
    private List<T> hand;
    private List<T> discard;

    public void addCard(T card) {
        cards.add(card);
    }

    public T draw() throws EmptyDeckException {
        if (cards.isEmpty()) {
            throw new EmptyDeckException("Deck is empty! Recycling discard pile...");
        }
        T drawn = cards.remove(0);
        hand.add(drawn);
        return drawn;
    }
}

```

Code example — items/Inventory.java:

```

public class Inventory<T extends Item> {
    private List<T> items;

    public void add(T item) throws OutOfStockException {
        if (items.size() >= maxCapacity)
            throw new OutOfStockException("Inventory full — cannot add " + item.getName());
        items.add(item);
    }
}

```

Listing 8: CoffeeDeck<T> and Inventory<T> — parametric polymorphism

Commentary: The bound `T extends CoffeeCard` is important — it guarantees every element in the deck has `getName()` and `getType()`, so I never need a cast or instanceof check when working with cards. The same principle applies to `Inventory<T extends Item>`. One generic class, compile-time safety, zero duplication.

3.7.3 Ad-hoc Polymorphism (Overloading)

Definition: Overloading allows multiple methods to share the same name but differ in their parameter lists. The compiler selects the correct version based on the argument types at the call site — this happens at compile time, not runtime.

Why it matters: Overloading improves readability. Instead of naming methods `drawOne()`, `drawMany()`, and `drawByType()`, I can use `draw()` for all three — the compiler picks the right one based on how it is called.

How I applied it: `CoffeeDeck<T>` has three overloads of `draw()` — one that draws a single card, one that draws n cards, and one that draws until a card of a specific type is found. `BattleEngine` uses all three at different points in the battle loop.

Code example — battle/CoffeeDeck.java:


```

public T draw() throws EmptyDeckException {
    if (cards.isEmpty()) {
        throw new EmptyDeckException("Deck is empty! Recycling discard pile...");
    }
    T drawn = cards.remove(0);
    hand.add(drawn);
    return drawn;
}

public List<T> draw(int n) throws EmptyDeckException {
    List<T> drawn = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        drawn.add(draw());
    }
    return drawn;
}

public List<T> draw(CardType filter) throws EmptyDeckException {
    List<T> result = new ArrayList<>();
    while (!cards.isEmpty()) {
        T card = cards.remove(0);
        hand.add(card);
        result.add(card);
        if (card.getType() == filter) break;
    }
    if (result.isEmpty()) {
        throw new EmptyDeckException("No cards of type " + filter + " found!");
    }
    return result;
}

```

Listing 9: CoffeeDeck.java — three overloads of draw(), selected by argument type at compile time

Commentary: Three methods share the same name but differ in their parameter lists — the compiler picks the correct one based on the argument type at the call site. `draw()` is called during the turn loop to draw one card, `draw(int n)` is called at battle start to draw the opening hand, and `draw(CardType filter)` draws until a specific card type is found. Same method name, three different behaviours, all resolved at compile time with no runtime cost.

3.7.4 Coercion Polymorphism (Type Casting / Interface Widening)

Definition: Coercion converts a reference from one type to another — either upcasting (subtype to supertype, always safe) or downcasting (supertype to subtype, must be verified at runtime).

Why it matters: Coercion allows code to work at a general level (GameRule receives a generic GameEvent) while still accessing specific details when needed (casting to BattleContext to get the current turn number or player health). It is the bridge between polymorphic interfaces and concrete implementations. This also gives the rule system room to grow — if I ever add a non-battle context (a shop event, an overworld event), GameRule wouldn't need to change at all, just a new class implementing GameEvent. Right now BattleContext is the only implementation, so every cast in the project is safe by construction, not just by luck.

How I applied it: Two places in the project use deliberate coercion. First, every GameRule implementation receives a GameEvent reference but casts it to BattleContext to access battle-specific state. Second, Barista.getEquipmentBonus() iterates over List<Item> and casts to Equipment when it finds an equipped item.

Code example — battle/ConditionalBonusRule.java:

```
@Override
public boolean appliesTo(GameEvent event) {
    BattleContext context = (BattleContext) event;
    return context.getPlayer().getHealth() < healthThreshold && !triggered;
}

@Override
public void execute(GameEvent event) {
    BattleContext context = (BattleContext) event;
    context.getPlayer().restoreStamina(bonusStamina);
    triggered = true;
    System.out.println(" Rule triggered: " + context.getPlayer().getName() +
        " gains " + bonusStamina + " emergency stamina!");
}
```

Code example — model/Barista.java:

```
public int getEquipmentBonus() {
    int bonus = 0;
    for (Item item : inventory.getAll()) {
        if (item instanceof items.Equipment) {
            items.Equipment eq = (items.Equipment) item;
            if (eq.isEquipped()) bonus += eq.getPowerBonus();
        }
    }
    return bonus;
}
```

Listing 10: Coercion in ConditionalBonusRule and Barista.getEquipmentBonus()

Commentary: Both casts are safe and deliberate. The `GameEvent` → `BattleContext` downcast is safe because `BattleEngine` always passes a `BattleContext` — the `GameEvent` interface exists so that future non-battle contexts could be added without changing the `GameRule` contract. The `Item` → `Equipment` downcast is guarded by an `instanceof` check first, so it never fails at runtime. Both cases show coercion being used responsibly: to access specialised behaviour at an interface boundary, without exposing that specialisation in the general contract.

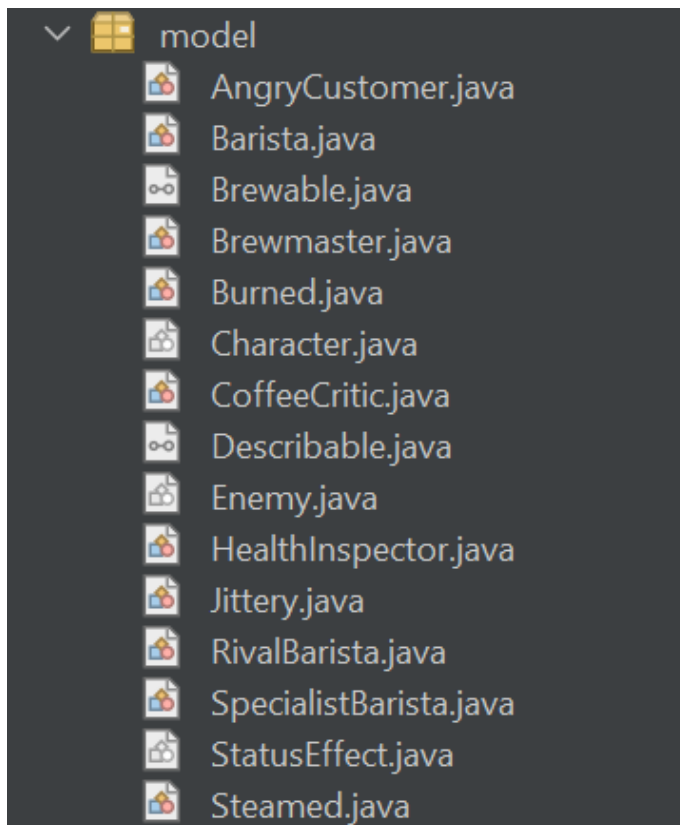
3.8 Modularity

Definition: Modularity is the organisation of code into independent, cohesive units — packages in Java — each with a single well-defined responsibility. Modules communicate through narrow, well-defined interfaces.

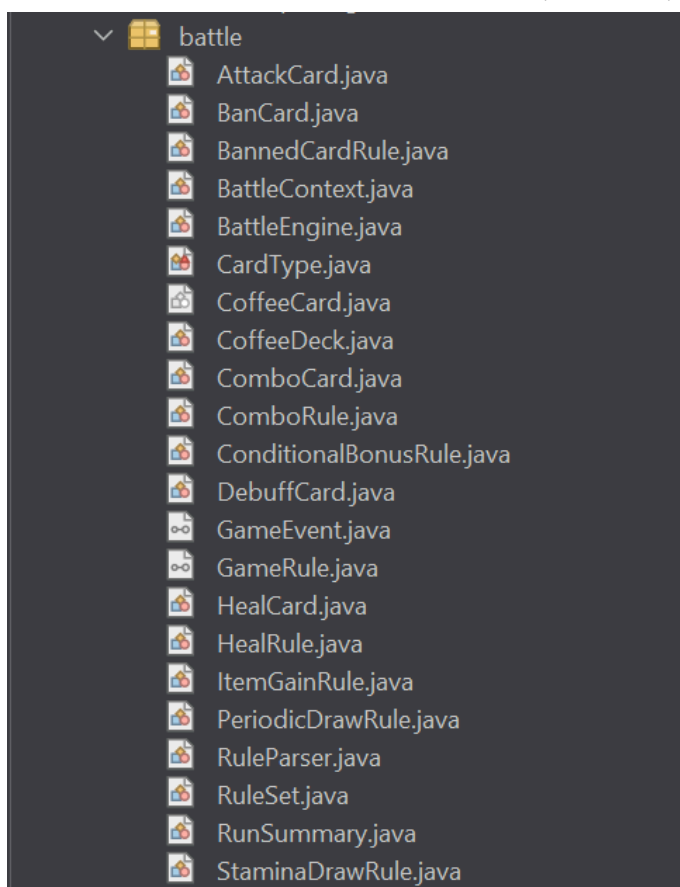
Why it matters: Modular code is easier to understand, test, and maintain. A bug in the battle package cannot silently corrupt state in the world package if the two packages don't know about each other's internals.

How I applied it: I organised the codebase into five packages, each with one responsibility:

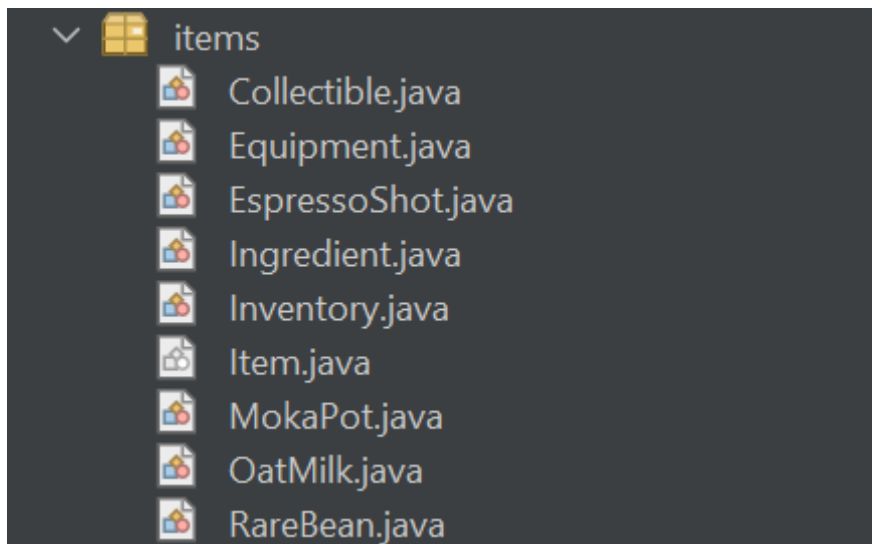
- **model** — what a Character IS and what state it carries (health, stamina, status effects), independent of how battles are conducted.



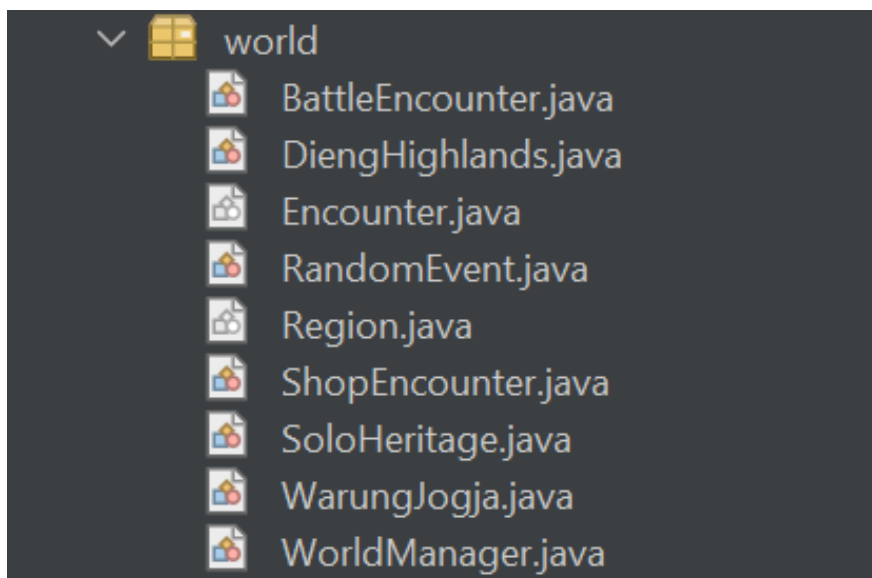
- **battle** — how a battle UNFOLDS: cards, the deck, the engine, and the rule system.



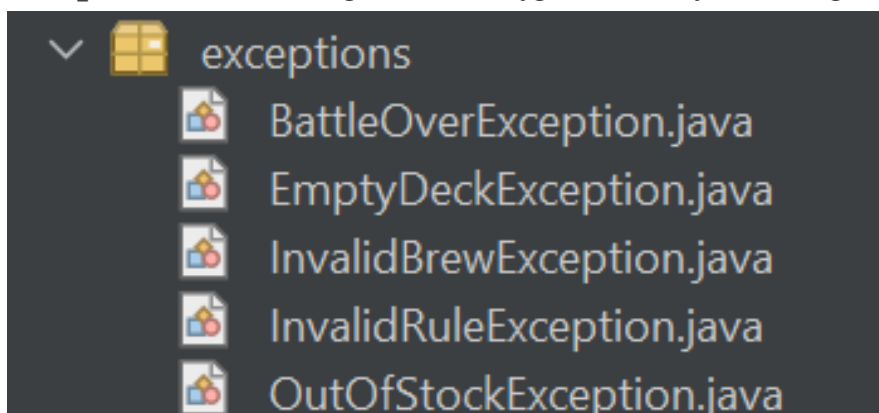
- **items** — what can be carried and consumed outside of battle.



- **world** — how the overworld is structured: regions, encounters, and progression.

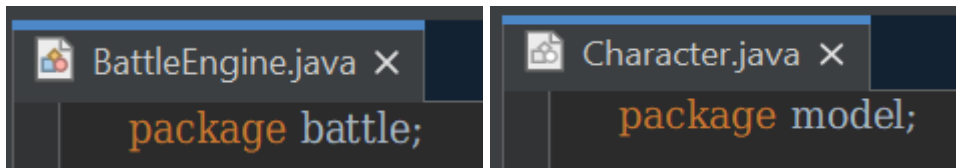


- **exceptions** — domain-specific error types shared by all other packages.



Listing 11 (Modularity): Five-package structure — one responsibility per package

Each class declares exactly one package at the top of the file:



Commentary: The most important consequence of this structure is that the packages are genuinely independent. The battle package imports from model and exceptions, but has zero knowledge of world or items internals. The items package has no knowledge of GameRule or BattleEngine at all — ItemGainRule in battle constructs Ingredient objects from items, but never the other way around. When I was debugging status effect bugs in model, I never had to worry about breaking world or items. That one-directional dependency is what made the project manageable at 61 Java files.

3.9 Composition

Definition: Composition models a HAS-A relationship — one object contains references to other objects as fields, rather than inheriting from them. The composed objects are independent and can be replaced or extended without changing the containing class.

Why it matters: Composition is more flexible than inheritance. If Barista inherited from a DeckOwner class, it would be locked into that structure permanently. By instead containing a CoffeeDeck as a field, I can change, extend, or replace the deck without touching Barista's inheritance chain.

How I applied it: Barista HAS-A CoffeeDeck and HAS-A Inventory. Region HAS-A List of Encounters. RuleSet HAS-A List of GameRules. None of these are inheritance relationships — they are ownership relationships, where one object delegates part of its behaviour to another.

Code example — model/Barista.java:

```
public class Barista extends Character {  
  
    private CoffeeDeck<CoffeeCard> deck;  
    private Inventory<Item> inventory;  
    private int beans;  
  
    public Barista(String name, int health, int stamina) {  
        super(name, health, stamina);  
        this.deck = new CoffeeDeck<>();  
        this.inventory = new Inventory<>();  
        this.beans = 0;  
    }  
}
```

Code example — world/Region.java:

```
public abstract class Region {
    private String name;
    private List<Encounter> encounterPool;

    public Region(String name) {
        this.name = name;
        this.encounterPool = new ArrayList<>();
    }

    public abstract void buildEncounterPool(battle.RuleSet ruleSet);

    public void shuffle() { Collections.shuffle(encounterPool); }

    public Encounter nextEncounter() { return encounterPool.remove(0); }

    protected void addEncounter(Encounter e) { encounterPool.add(e); }
```

Listing 12: Barista.java — composition over inheritance for deck and inventory

Commentary: Barista does not inherit from DeckOwner or InventoryOwner — it simply contains a deck and an inventory. The same principle applies to the rule system: RuleSet HAS-A List<GameRule>, injected into BattleEngine at construction. BattleEngine never needs to know how many rules exist or what they do — it just calls `appliesTo()` and `execute()` on each one. Composition is what makes this possible: the rule list can grow at runtime as the player types new rules, and BattleEngine adapts automatically without any changes.

3.10 Hierarchy

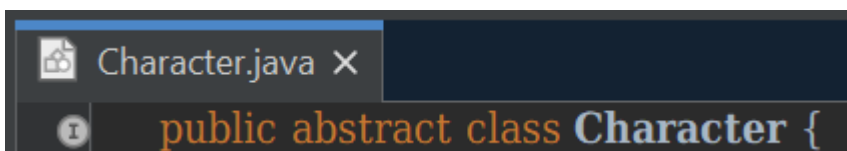
Definition: A hierarchy is a structured, tree-like organisation of classes produced by inheritance. Type hierarchies enable polymorphic dispatch — code can operate on a supertype without knowing which concrete subtype it is dealing with.

Why it matters: Hierarchies allow code to work at multiple levels of abstraction. BattleEngine can fight any Enemy without knowing whether it is RivalBarista, HealthInspector, or Brewmaster. New Enemy types can be added without modifying BattleEngine at all.

How I applied it: Three parallel hierarchies structure the project:

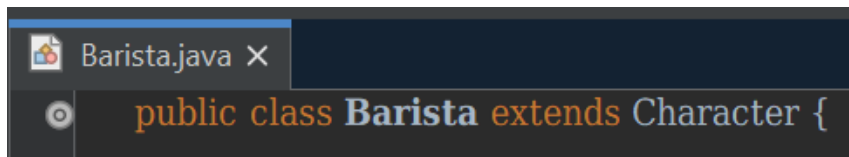
- **Character hierarchy** — Character → Barista → SpecialistBarista (three levels), and Character → Enemy → five concrete enemy types.

// model/Character.java

A screenshot of an IDE window titled 'Character.java'. The code visible is the beginning of a Java class: 'public abstract class Character {'.

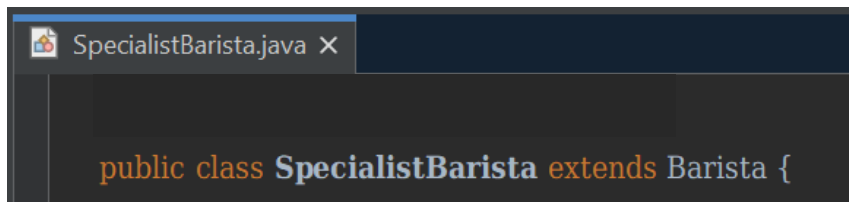
```
public abstract class Character {
```

// model/Barista.java



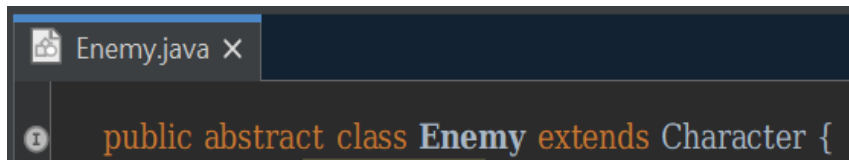
```
public class Barista extends Character {
```

// model/SpecialistBarista.java



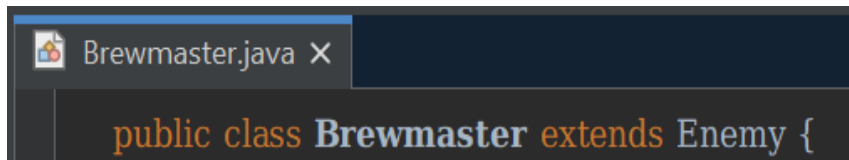
```
public class SpecialistBarista extends Barista {
```

// model/Enemy.java



```
public abstract class Enemy extends Character {
```

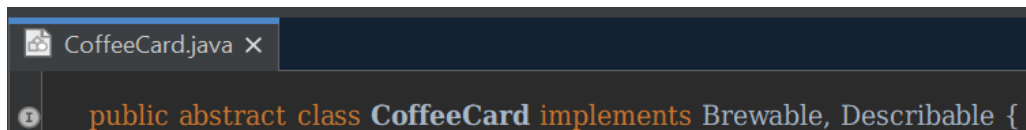
// model/Brewmaster.java



```
public class Brewmaster extends Enemy {
```

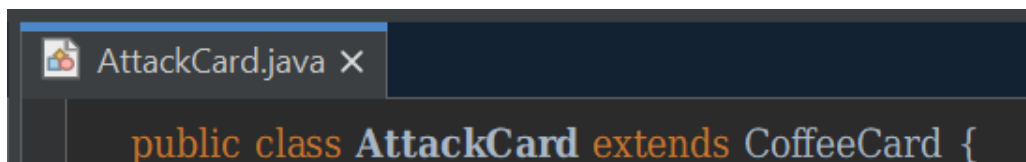
- **CoffeeCard hierarchy** — CoffeeCard → AttackCard, HealCard, DebuffCard, ComboCard, BanCard (five subtypes).

// battle/CoffeeCard.java



```
public abstract class CoffeeCard implements Brewable, Describable {
```

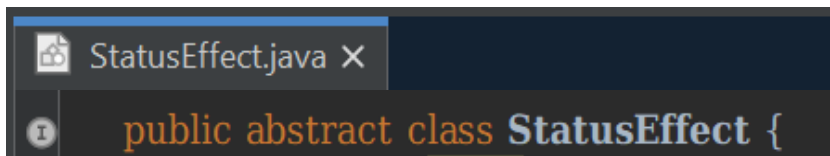
// battle/AttackCard.java



```
public class AttackCard extends CoffeeCard {
```

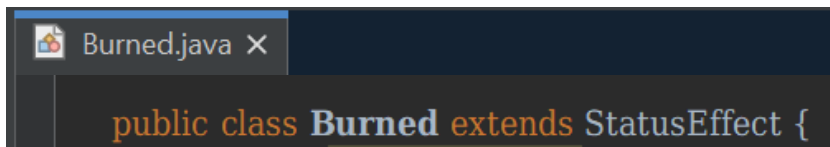

- **StatusEffect hierarchy** — StatusEffect → Burned, Steamed, Jittery.

// model/StatusEffect.java



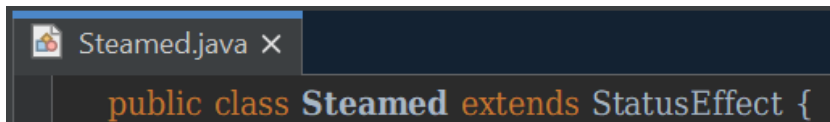
```
public abstract class StatusEffect {
```

// model/Burned.java



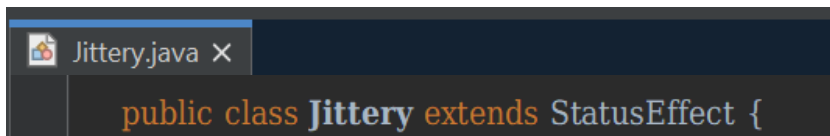
```
public class Burned extends StatusEffect {
```

// model/Steamed.java



```
public class Steamed extends StatusEffect {
```

// model/Jittery.java



```
public class Jittery extends StatusEffect {
```

Listing 13 (Hierarchy): Three parallel class hierarchies in Java Coffee

Additional parallel hierarchies: Region → three regions; Encounter → three encounter types; Item → Ingredient, Equipment, Collectible.

Commentary: Three parallel hierarchies structure the project. The StatusEffect hierarchy is the clearest example — the abstract base declares only `tick()`, `isExpired()`, and `decrementTurn()`. Burned, Steamed, and Jittery each implement `tick()` completely differently — dealing damage, draining stamina, and rolling a 50% fumble chance respectively — yet `Character.tickStatusEffects()` iterates over `List<StatusEffect>` and calls `tick()` identically for all three. I never had to add a special case for any status effect type. That is hierarchy-based polymorphism working exactly as intended.

3.11 Reuse

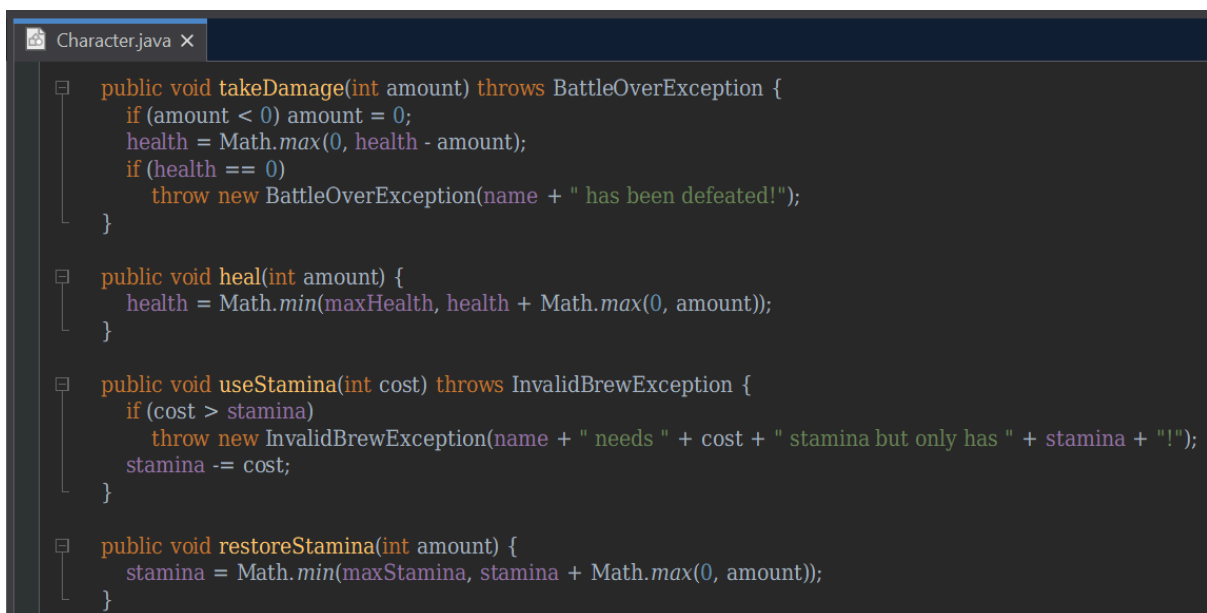
Definition: Code reuse means writing something once and using it in multiple places, avoiding duplication. In Java, reuse is achieved through inheritance, interfaces, generics, and composition.

Why it matters: Reuse reduces duplication, bugs, and maintenance effort. If `takeDamage()` is written once in `Character` and inherited by all entities, fixing a bug in `takeDamage()` fixes it everywhere — in `Barista`, `RivalBarista`, `Brewmaster`, and every future enemy type.

How I applied it:

- **INHERITANCE REUSE** — `takeDamage()`, `heal()`, `useStamina()`, `restoreStamina()` → written once in `Character`, inherited by all 7 character types

Code example — model/Character.java:



```
Character.java X
public void takeDamage(int amount) throws BattleOverException {
    if (amount < 0) amount = 0;
    health = Math.max(0, health - amount);
    if (health == 0)
        throw new BattleOverException(name + " has been defeated!");
}

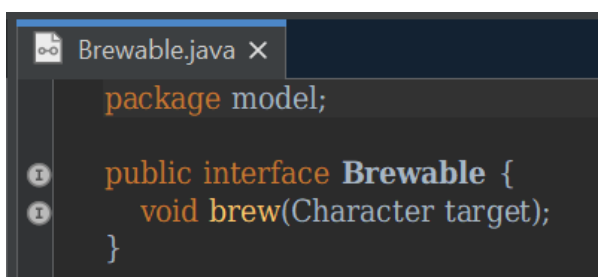
public void heal(int amount) {
    health = Math.min(maxHealth, health + Math.max(0, amount));
}

public void useStamina(int cost) throws InvalidBrewException {
    if (cost > stamina)
        throw new InvalidBrewException(name + " needs " + cost + " stamina but only has " + stamina + "!");
    stamina -= cost;
}

public void restoreStamina(int amount) {
    stamina = Math.min(maxStamina, stamina + Math.max(0, amount));
}
```

- **INTERFACE REUSE** — `Brewable` implemented by `CoffeeCard` AND `Item` → same contract, two unrelated hierarchies

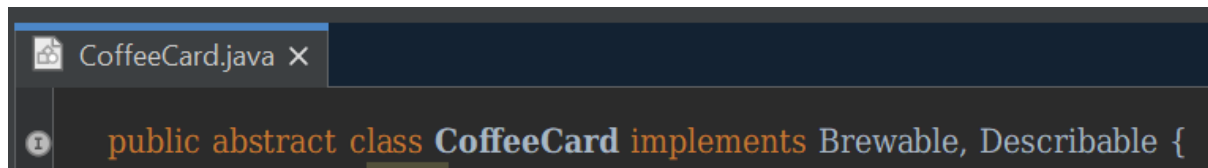
Code example — model/Brewable.java + first lines of CoffeeCard.java and Item.java:



```
Brewable.java X
package model;

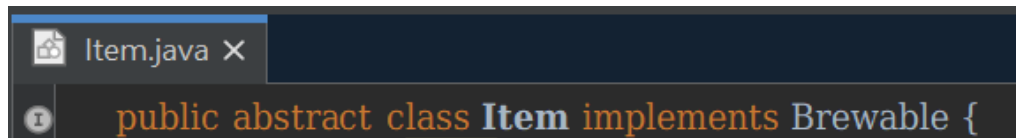
public interface Brewable {
    void brew(Character target);
}
```

// battle/CoffeeCard.java



```
public abstract class CoffeeCard implements Brewable, Describable {
```

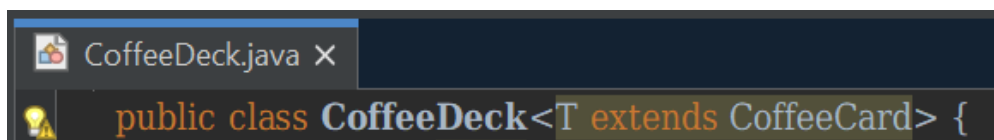
// items/Item.java



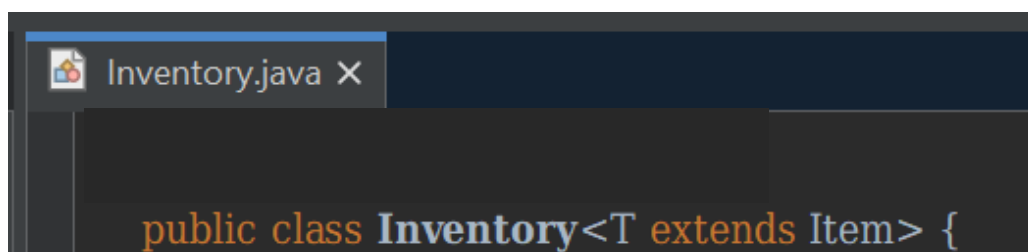
```
public abstract class Item implements Brewable {
```

- **GENERIC REUSE** — `CoffeeDeck<T extends CoffeeCard>` works for any card type. `Inventory<T extends Item>` works for any item type → one implementation, many types

Code example — battle/CoffeeDeck.java + items/Inventory.java first lines:



```
public class CoffeeDeck<T extends CoffeeCard> {
```



```
public class Inventory<T extends Item> {
```

Listing 14 (Reuse): *Character.java* — four methods written once, inherited by all character types

Commentary: Reuse appears at four levels in the project. Inheritance reuse — `takeDamage()`, `heal()`, `useStamina()`, and `restoreStamina()` are written once in `Character` and inherited by all seven concrete character types without duplication. Interface reuse — `Brewable` is implemented by both `CoffeeCard` and `Item`, two completely unrelated hierarchies, so the same brew logic works for both. Generic reuse — `CoffeeDeck<T>` and `Inventory<T>` are written once and work safely for any valid type. Composition reuse — the `GameRule` evaluation loop in `BattleEngine` is reused for all seven `GameRule` implementations — the six player-authored types and `BannedCardRule` and will work for any future rule type without changes. Getting that boundary right between what belongs in the supertype and what belongs in the subtype is what made the reuse work cleanly.

3.12 Extensibility

Definition: Extensibility is the ability to add new behaviour without modifying existing code — the Open/Closed Principle: open for extension, closed for modification.

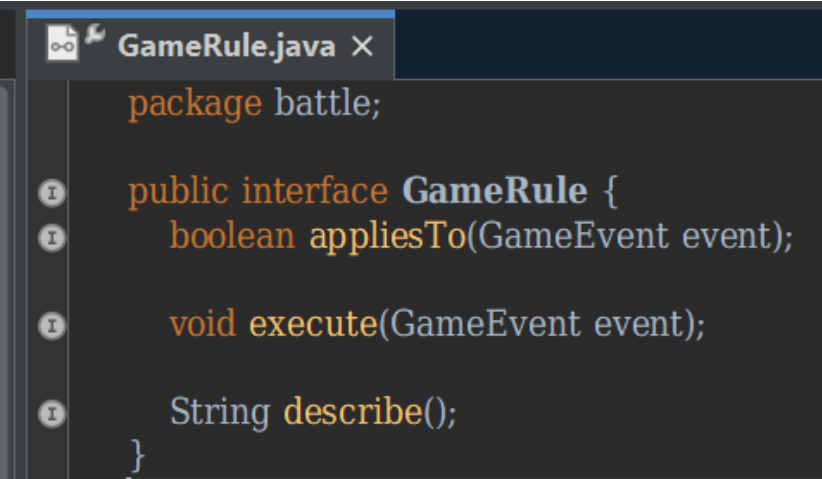
Why it matters: Extensible systems can grow safely. New features can be added without risk of breaking existing tested code. In Java Coffee this matters because the professor's feedback specifically required that the rule system be genuinely extensible — new rule types, not just new parameters.

How I applied it: The GameRule interface is the primary example. To add a new rule type, I just create a new class implementing GameRule. BattleEngine, RuleSet, and all existing rule classes need zero changes. The same principle applies throughout the project:

- New card types — extend CoffeeCard and implement `use()`. BattleEngine needs no changes.
- New enemy types — extend Enemy and implement `decideAction()`. BattleEngine needs no changes.
- New regions — extend Region and implement `buildEncounterPool()`. WorldManager needs no changes.
- New status effects — extend StatusEffect and implement `tick()`. Character.tickStatusEffects() handles them automatically.

CLOSED FOR MODIFICATION

Code example — battle/GameRule.java:



```
package battle;

public interface GameRule {
    boolean appliesTo(GameEvent event);

    void execute(GameEvent event);

    String describe();
}
```

OPEN FOR EXTENSION

Code example — battle/GameRule.java:

```

ItemGainRule.java ×
public class ItemGainRule implements GameRule {
    private int healthThreshold;
    private String itemName;
    private boolean triggered;
}

```

Code example — battle/PeriodicDrawRule.java:

```

PeriodicDrawRule.java ×
public class PeriodicDrawRule implements GameRule {
    private int interval;
    private int cardsToGain;
}

```

→ new rule types added without touching GameRule or BattleEngine

Listing 15 (Extensibility): GameRule.java — the interface that makes the rule system extensible

Commentary: `GameRule` is closed for modification — I never changed it after writing it. But I added seven concrete rule types just by creating new classes that implement it, and `BattleEngine`, `RuleSet`, and all the existing rule classes needed zero changes. The same holds everywhere else: new card types extend `CoffeeCard`, new enemy types extend `Enemy`, new status effects extend `StatusEffect` — none of it touches existing code. `BattleEngine`'s `applyCustomRules()` loop hasn't changed since I first wrote it, even as the rule system grew from two rule types to seven. That's the payoff of designing for extensibility early.

3.13 Exception Handling

Definition: Exception handling deals with errors by throwing exception objects when something goes wrong, so the program can catch them and recover instead of crashing.

Why it matters: Exception handling separates error logic from normal logic, making code clearer. Without it, every method that could fail would need to return a special error value and every caller would need to check for it — making the code much harder to read and maintain.

How I applied it: I defined five custom checked exceptions, each representing a specific domain error:

- `EmptyDeckException` — thrown by `CoffeeDeck.draw()` when the draw pile and discard pile are both exhausted.
- `InvalidBrewException` — thrown by `Character.useStamina()` when a card costs more stamina than the player has.

- `OutOfStockException` — thrown by `Inventory<T>` when adding to a full inventory or looking up a missing item.
- `BattleOverException` — thrown by `Character.takeDamage()` when health reaches zero; propagates to end the battle cleanly.
- `InvalidRuleException` — thrown by `RuleParser` when the player types a rule that matches no supported pattern.

FIVE CUSTOM EXCEPTIONS

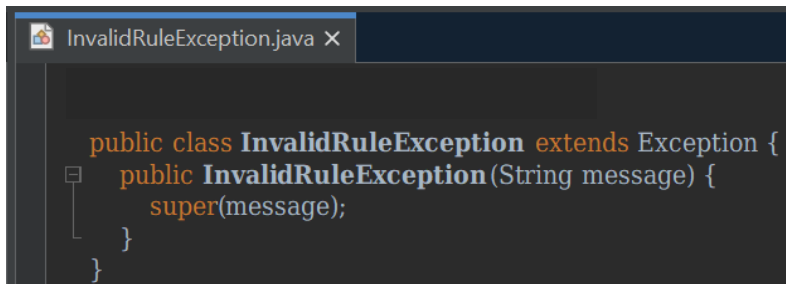
Code example — exceptions/ package — all five classes together:

```
BattleOverException.java ×  
  
public class BattleOverException extends Exception {  
    public BattleOverException(String message) {  
        super(message);  
    }  
}
```

```
InvalidBrewException.java ×  
  
public class InvalidBrewException extends Exception {  
    public InvalidBrewException(String message) {  
        super(message);  
    }  
}
```

```
EmptyDeckException.java ×  
  
public class EmptyDeckException extends Exception {  
    public EmptyDeckException(String message) {  
        super(message);  
    }  
}
```

```
OutOfStockException.java ×  
  
public class OutOfStockException extends Exception {  
    public OutOfStockException(String message) {  
        super(message);  
    }  
}
```



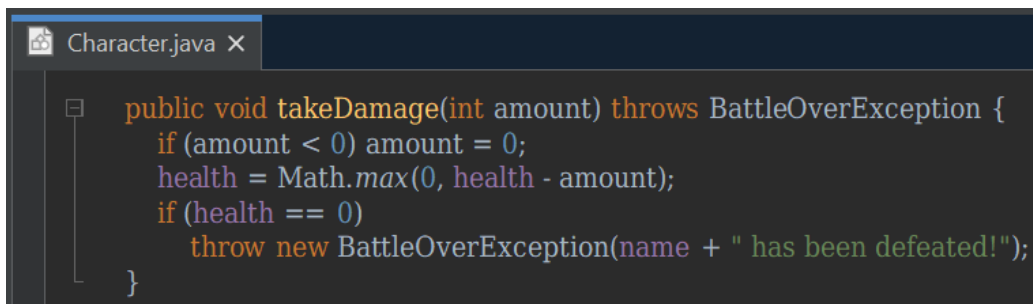
```

InvalidRuleException.java X
public class InvalidRuleException extends Exception {
    public InvalidRuleException(String message) {
        super(message);
    }
}

```

WHERE THROWN

Code example — model/Character.java:



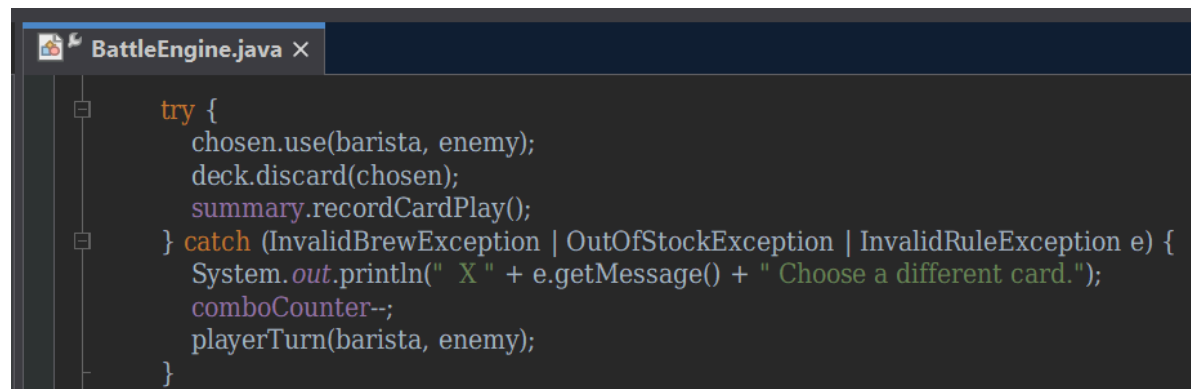
```

Character.java X
public void takeDamage(int amount) throws BattleOverException {
    if (amount < 0) amount = 0;
    health = Math.max(0, health - amount);
    if (health == 0)
        throw new BattleOverException(name + " has been defeated!");
}

```

RECOVERABLE — player re-prompted

Code example — battle/BattleEngine.java:



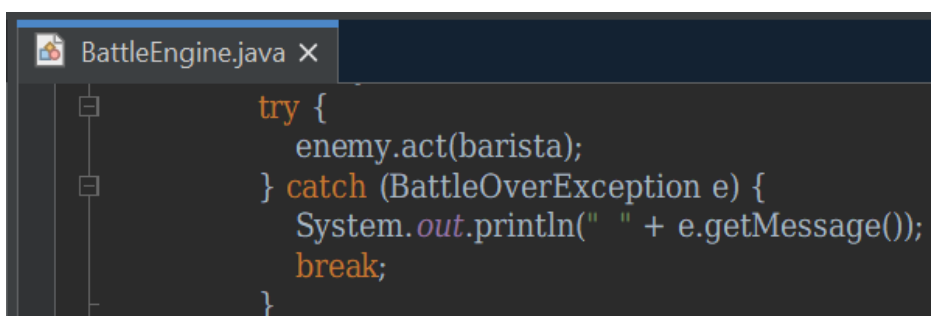
```

BattleEngine.java X
try {
    chosen.use(barista, enemy);
    deck.discard(chosen);
    summary.recordCardPlay();
} catch (InvalidBrewException | OutOfStockException | InvalidRuleException e) {
    System.out.println(" X " + e.getMessage() + " Choose a different card.");
    comboCounter--;
    playerTurn(barista, enemy);
}

```

FATAL — battle ends immediately

Code example — battle/BattleEngine.java:



```

BattleEngine.java X
try {
    enemy.act(barista);
} catch (BattleOverException e) {
    System.out.println(" " + e.getMessage());
    break;
}

```

Listing 16 (Exception Handling): Two different recovery strategies for two exception severities

Commentary: Five custom exceptions, each representing a specific domain error with a deliberately different severity. `InvalidBrewException` is recoverable within the same turn — the player is re-prompted to choose a different card. `BattleOverException` is fatal — it breaks out of the battle loop immediately and ends the battle. `InvalidRuleException` is caught at the console input boundary in `Main` and the player is asked to re-enter their rule. This deliberate separation of recoverable from fatal errors is what keeps the game state consistent throughout — the player never sees a crash, only a clear message and a chance to continue.

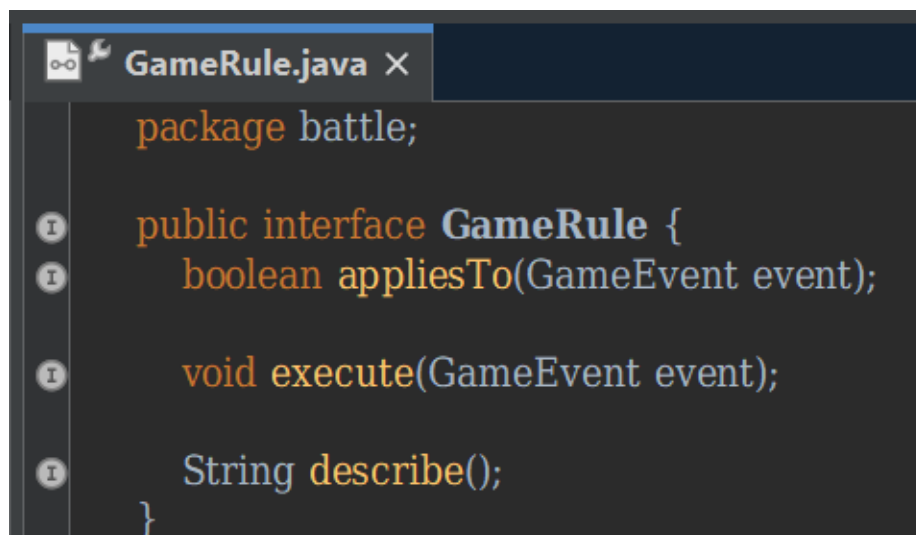
4. Player-Authored Game Rules

This section explains the central design contribution of the project — the IF/THEN rule system that lets the player create brand-new game behaviours at runtime. All code examples are taken directly from github.com/nasywaarr/barista-quest.

4.1 The GameRule Interface

At the centre of the system is a small interface that every rule must implement — whether it was typed by the player at setup or injected by an enemy mid-battle:

Application in the Project — `battle/GameRule.java`:



```
GameRule.java X
package battle;

public interface GameRule {
    boolean appliesTo(GameEvent event);

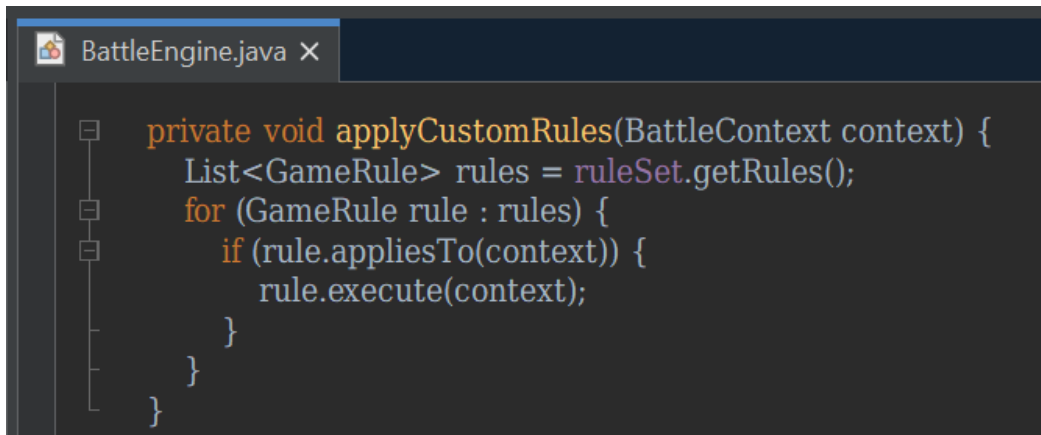
    void execute(GameEvent event);

    String describe();
}
```

Listing 17: `GameRule.java` — shared contract for every rule

Every turn, `BattleEngine` builds a `BattleContext` snapshot and iterates over `RuleSet`'s `List<GameRule>`, calling `appliesTo()` on each rule and — if it returns true — calling `execute()`. It doesn't matter whether the rule came from the player at setup or an enemy mid-battle — both are just `GameRule` objects in the same list, checked and run the same way.

Application in the Project — battle/BattleEngine.java:



```
private void applyCustomRules(BattleContext context) {
    List<GameRule> rules = ruleSet.getRules();
    for (GameRule rule : rules) {
        if (rule.appliesTo(context)) {
            rule.execute(context);
        }
    }
}
```

Listing 18: BattleEngine — every GameRule checked identically via polymorphic dispatch

Commentary: BattleEngine never checks what type of rule it is dealing with. It calls `appliesTo()` and `execute()` on every rule through the same `GameRule` interface reference — the JVM dispatches to the correct implementation at runtime. A rule typed by the player at setup and a ban injected by `HealthInspector` mid-battle are both treated identically by the engine. There is no special case for either.

4.2 Concrete Rule Implementations

I implemented seven concrete classes that all implement `GameRule`. Six of them are ones the player can author through `RuleParser`'s IF/THEN grammar. The seventh, `BannedCardRule`, is not player-facing — it's built only by enemies (`HealthInspector`, `CoffeeCritic`) to ban a card type mid-battle. Section 4.4 covers `BannedCardRule` separately since it works a bit differently.

- `ConditionalBonusRule` — fires once when HP drops below a threshold, restores stamina.
- `ItemGainRule` — fires once when HP drops below a threshold, adds an item to inventory.
- `HealRule` — fires once when HP drops below a threshold, restores HP directly.
- `StaminaDrawRule` — fires once when stamina drops below a threshold, draws extra cards.
- `PeriodicDrawRule` — fires every N turns, draws extra cards. Repeating — never expires.
- `ComboRule` — fires when the combo counter reaches N, boosts damage for that turn.

Despite having different fields, different conditions, and different effects, all six implement the same three-method `GameRule` interface. `BattleEngine` never needs to know which one it is calling.

Application in the Project — `battle/ConditionalBonusRule.java`:

```
package battle;

public class ConditionalBonusRule implements GameRule {
    private int healthThreshold;
    private int bonusStamina;
    private boolean triggered;

    @Override
    public boolean appliesTo(GameEvent event) {
        BattleContext context = (BattleContext) event;
        return context.getPlayer().getHealth() < healthThreshold && !triggered;
    }

    @Override
    public void execute(GameEvent event) {
        BattleContext context = (BattleContext) event;
        context.getPlayer().restoreStamina(bonusStamina);
        triggered = true;
        System.out.println(" Rule triggered: " + context.getPlayer().getName() +
            " gains " + bonusStamina + " emergency stamina!");
    }
}
```

Application in the Project — `battle/PeriodicDrawRule.java`:

```
public class PeriodicDrawRule implements GameRule {
    private int interval;
    private int cardsToGain;

    @Override
    public boolean appliesTo(GameEvent event) {
        BattleContext context = (BattleContext) event;
        return context.getCurrentTurn() > 0 && context.getCurrentTurn() % interval == 0;
    }
}
```

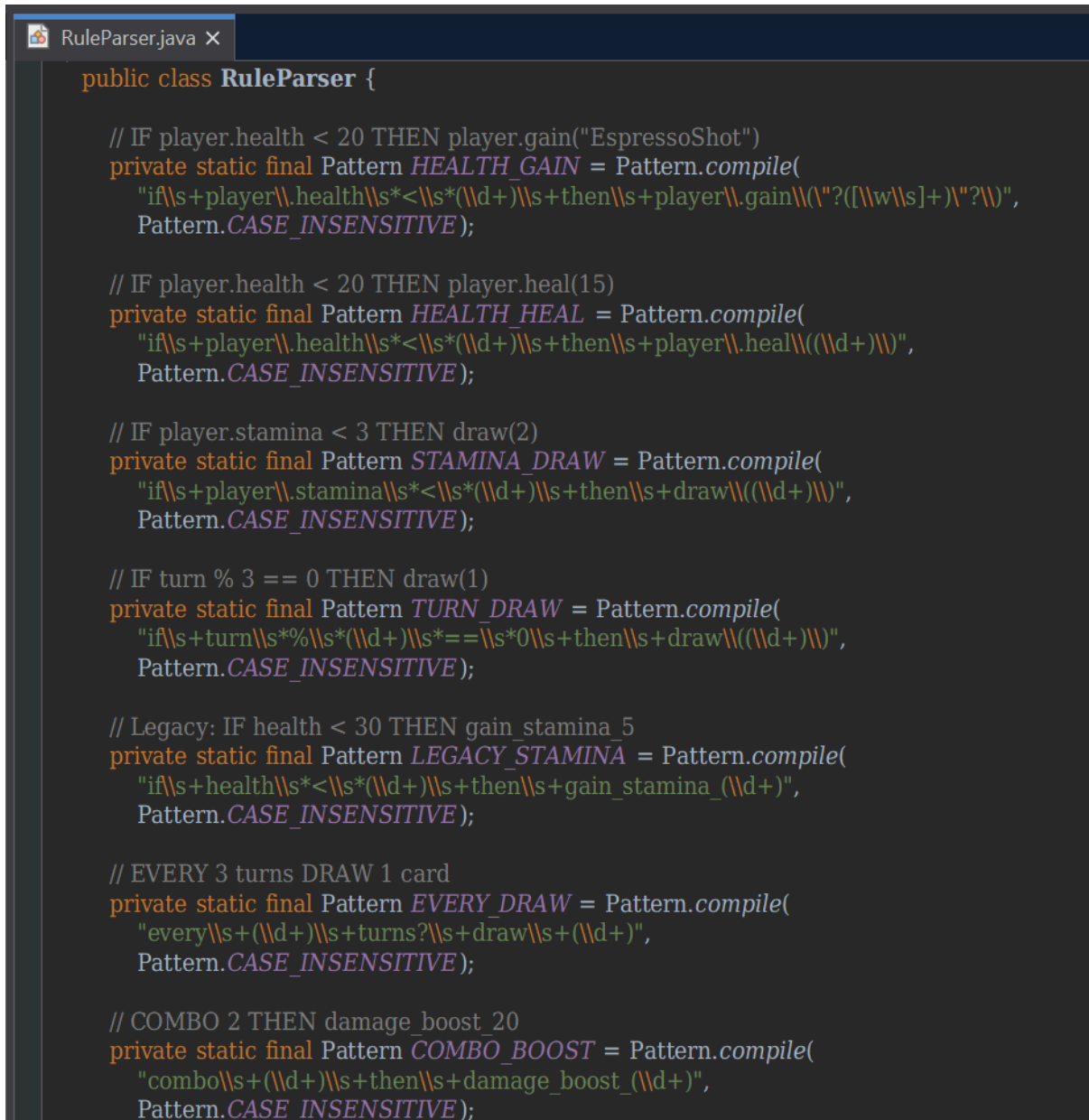
Listing 19: Two concrete `GameRule` implementations with different internal shapes

Commentary: The most interesting structural difference between the six rule classes is the `triggered` flag. `ConditionalBonusRule`, `ItemGainRule`, `HealRule`, and `StaminaDrawRule` are all one-shot — they fire once and never again, enforced by setting `triggered = true` in `execute()` and checking `!triggered` in `appliesTo()`. `PeriodicDrawRule` and `ComboRule` are repeating — they have no `triggered` flag and can fire every time their condition is met. This difference in internal shape is completely invisible to `BattleEngine` — it just calls `appliesTo()` and `execute()` on all six identically.

4.3 The IF/THEN RuleParser

At game setup, the player is prompted to type rules in plain text. `RuleParser.parseRules()` splits the input on semicolons, then matches each rule against six compiled `java.util.regex.Pattern` instances and constructs the appropriate `GameRule` object:

Application in the Project — `battle/RuleParser.java`:



```
public class RuleParser {

    // IF player.health < 20 THEN player.gain("EspressoShot")
    private static final Pattern HEALTH_GAIN = Pattern.compile(
        "if\\s+player\\.health\\s*<\\s*(\\d+)\\s+then\\s+player\\.gain\\s*\\s*\\s*\\s*\\s*\\s*\\s*\\s*",
        Pattern.CASE_INSENSITIVE);

    // IF player.health < 20 THEN player.heal(15)
    private static final Pattern HEALTH_HEAL = Pattern.compile(
        "if\\s+player\\.health\\s*<\\s*(\\d+)\\s+then\\s+player\\.heal\\s*\\s*\\s*\\s*\\s*\\s*",
        Pattern.CASE_INSENSITIVE);

    // IF player.stamina < 3 THEN draw(2)
    private static final Pattern STAMINA_DRAW = Pattern.compile(
        "if\\s+player\\.stamina\\s*<\\s*(\\d+)\\s+then\\s+draw\\s*\\s*\\s*\\s*\\s*\\s*",
        Pattern.CASE_INSENSITIVE);

    // IF turn % 3 == 0 THEN draw(1)
    private static final Pattern TURN_DRAW = Pattern.compile(
        "if\\s+turn\\s*%\\s*(\\d+)\\s+then\\s+draw\\s*\\s*\\s*\\s*\\s*\\s*",
        Pattern.CASE_INSENSITIVE);

    // Legacy: IF health < 30 THEN gain_stamina 5
    private static final Pattern LEGACY_STAMINA = Pattern.compile(
        "if\\s+health\\s*<\\s*(\\d+)\\s+then\\s+gain_stamina_\\s*(\\d+)",
        Pattern.CASE_INSENSITIVE);

    // EVERY 3 turns DRAW 1 card
    private static final Pattern EVERY_DRAW = Pattern.compile(
        "every\\s+(\\d+)\\s+turns?\\s+draw\\s+(\\d+)",
        Pattern.CASE_INSENSITIVE);

    // COMBO 2 THEN damage_boost_20
    private static final Pattern COMBO_BOOST = Pattern.compile(
        "combo\\s+(\\d+)\\s+then\\s+damage_boost_\\s*(\\d+)",
        Pattern.CASE_INSENSITIVE);
```

Listing 20: `RuleParser.java` — six compiled regex patterns covering all supported syntax

For example, when the player types `IF player.health < 30 THEN player.gain("OatMilk")`, the `HEALTH_GAIN` pattern matches and extracts two values — `threshold=30` and `itemName="OatMilk"`. The parser then constructs a new `ItemGainRule(30, "OatMilk")` and adds it to `RuleSet`. From that point forward,

every turn, BattleEngine checks whether the player's HP is below 30 — if so, `execute()` creates a fresh Ingredient of type OAT_MILK and adds it to the barista's inventory. That object did not exist before the player typed that line.

The six patterns cover the full grammar:

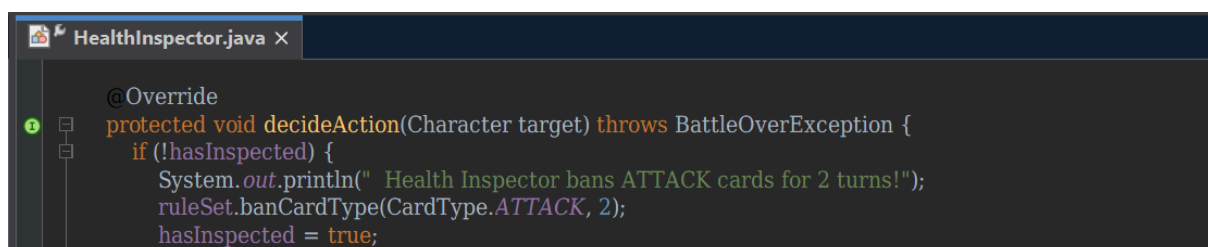
- **ConditionalBonusRule** – IF health < N THEN gain_stamina_N
- **ItemGainRule** – IF player.health < N THEN
player.gain("ItemName")
- **HealRule** – IF player.health < N THEN player.heal(N)
- **StaminaDrawRule** – IF player.stamina < N THEN draw(N)
- **PeriodicDrawRule** – IF turn % N == 0 THEN draw(N) or EVERY N
turns DRAW N card
- **ComboRule** – COMBO N THEN damage_boost_N

Commentary: The regex approach means the grammar is fixed — the player cannot type arbitrary Java — but within the grammar, the values are completely free. The player decides the threshold (any number), the item (any valid item name), the interval (any number of turns). This is what makes the rules genuinely non-parametric: the player is not picking from a menu, they are authoring the rule from scratch. RuleParser just reads what they typed and turns it into a live Java object.

4.4 Dynamic Rule Mutation by Enemies

One of the requirements from the proposal feedback was that rules should be first-class objects usable by any entity — not just the player at setup. Both HealthInspector and CoffeeCritic satisfy this the same way: each builds an actual GameRule object (BannedCardRule) and adds it to RuleSet's rule list with `addRule()`, so BattleEngine evaluates it every turn exactly like a rule the player typed themselves. HealthInspector bans ATTACK cards, CoffeeCritic bans HEAL cards — different card types, identical mechanism underneath.

Application in the Project — model/HealthInspector.java:



```
HealthInspector.java x
@Override
protected void decideAction(Character target) throws BattleOverException {
    if (!hasInspected) {
        System.out.println(" Health Inspector bans ATTACK cards for 2 turns!");
        ruleSet.banCardType(CardType.ATTACK, 2);
        hasInspected = true;
    }
}
```

Application in the Project — model/CoffeeCritic.java:

```
CoffeeCritic.java x
@Override
protected void decideAction(Character target) throws BattleOverException {
    if (!hasReviewed) {
        System.out.println(" Coffee Critic bans HEAL cards for 2 turns!");
        ruleSet.addRule(new BannedCardRule(ruleSet, CardType.HEAL, 2));
        hasReviewed = true;
    }
}
```

Application in the Project — battle/BannedCardRule.java:

```
BannedCardRule.java x
private final CardType bannedType;
private final int turns;
private boolean applied;

public BannedCardRule(RuleSet ruleSet, CardType bannedType, int turns) {
    this.ruleSet = ruleSet;
    this.bannedType = bannedType;
    this.turns = turns;
}

@Override
public boolean appliesTo(GameEvent event) {
    return !applied;
}

@Override
public void execute(GameEvent event) {
    ruleSet.banCardType(bannedType, turns);
    applied = true;
}
```

Application in the Project — battle/RuleSet.java:

```
RuleSet.java x
private Map<CardType, Integer> bannedCardDurations;

public void banCardType(CardType type, int turns) {
    bannedCardDurations.put(type, turns);
}

public void tick() {
    Iterator<Map.Entry<CardType, Integer>> it = bannedCardDurations.entrySet().iterator();
    while (it.hasNext()) {
        Map.Entry<CardType, Integer> entry = it.next();
        int remaining = entry.getValue() - 1;
        if (remaining <= 0) {
            it.remove();
        } else {
            entry.setValue(remaining);
        }
    }
}

public boolean isCardAllowed(CardType type) {
    return allowedCardTypes.contains(type) && !bannedCardDurations.containsKey(type);
}
```

Listing 21: Both enemies inject the same GameRule object; RuleSet tracks each ban's countdown independently.

Commentary: What I like most about this design is that a player-authored rule ("gain OatMilk when HP is below 30") and an enemy-triggered ban ("no ATTACK cards for 2 turns") are managed by the exact same `RuleSet`. `BattleEngine` checks both through the same `applyCustomRules()` call, and `isCardAllowed()` controls the player's hand either way — there's no special case for "player rule vs. enemy rule." There isn't even a special case between enemies: `HealthInspector` and `CoffeeCritic` both ban cards through the identical `addRule(new BannedCardRule(...))` path, and `banCardType()` is only ever called from inside `BannedCardRule.execute()` — one code path, used by everyone. `RuleSet` also tracks each banned type's duration separately in a `Map<CardType, Integer>`, so two bans starting on different turns expire independently instead of sharing one timer.

5. UML Diagrams

The diagrams in this section come from the actual implementation at github.com/nasywaarr/barista-quest. I used standard UML notation: a hollow triangle for extends/implements (IS-A), and a filled diamond for composition (HAS-A).

5.1 Model Package

Figure 1 shows the model package — the full Character hierarchy with two parallel branches. On the left, `Character` → `Barista` → `SpecialistBarista` forms a three-level chain. On the right, `Character` → `Enemy` → five concrete enemy types forms a parallel branch.

The `StatusEffect` hierarchy sits separately, connected to `Character` only through composition — a `Character` HAS-A `List<StatusEffect>`, not IS-A `StatusEffect`. This is a deliberate design choice: because there is no inheritance relationship between `Character` and `StatusEffect`, I can add new status effects without ever modifying `Character`. The `tickStatusEffects()` loop handles them all automatically through polymorphic dispatch.

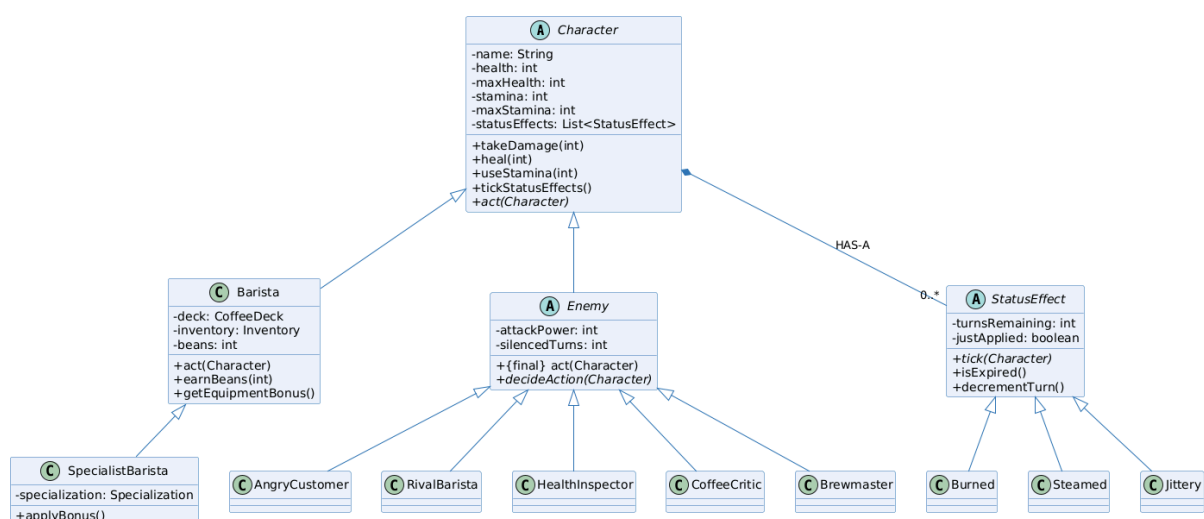


Figure 1: Class diagram of the model package

5.2 Battle Package

Figure 2 shows the battle package split into two halves.

The left half shows the `CoffeeCard` hierarchy: five subtypes (`AttackCard`, `HealCard`, `DebuffCard`, `ComboCard`, `BanCard`) extending the abstract `CoffeeCard`. `CoffeeDeck<T>` holds three separate lists internally — draw pile, hand, and discard pile — which is what lets cards get recycled without duplicates.

The right half shows the `GameRule` system. `RuleParser` reads what the player types and builds the matching rule object — `ItemGainRule`, `PeriodicDrawRule`, `ComboRule`, and so on. Enemies add rules the same way, by building a `BannedCardRule` and calling `addRule()`. Every rule, whether it came from the player at setup or an enemy mid-battle, implements the same `GameRule` interface and lives in the same list inside `RuleSet`. `BattleEngine` just loops over that list once per turn.

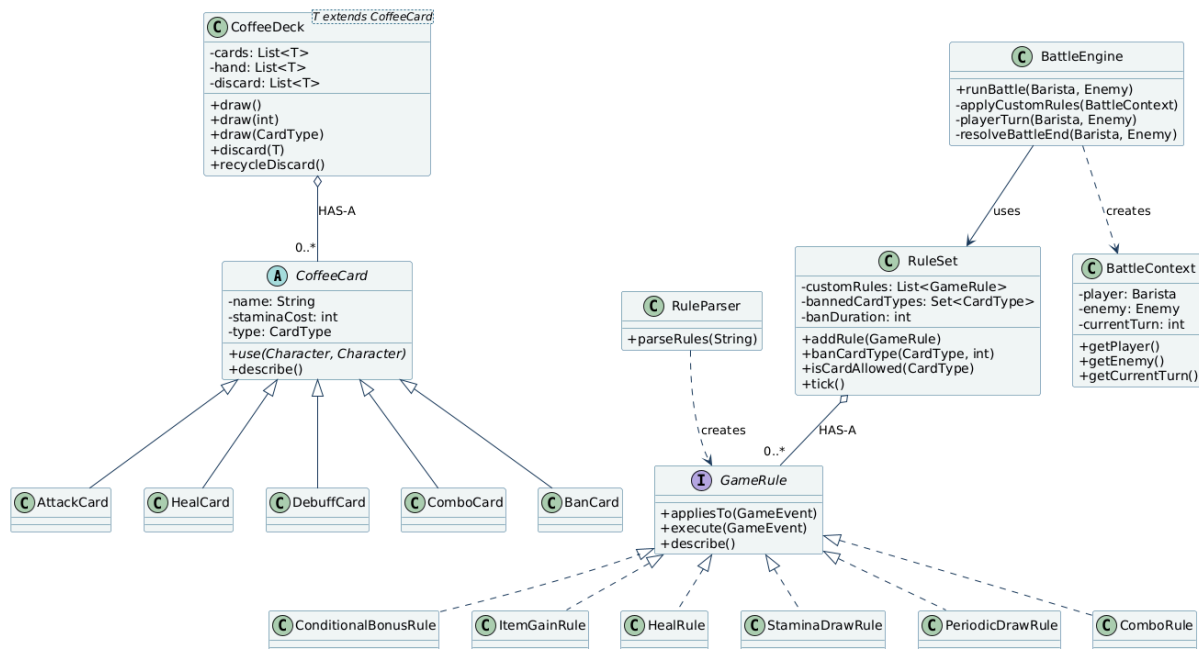


Figure 2: Class diagram of the battle package and `GameRule` system

5.3 Items and World Packages

Figures 3 and 4 cover the two remaining packages.

Figure 3 shows the items package. `Item` is the abstract root and implements `Brewable`, so every item can be used on a `Character` through the same `brew()` method. Three category classes extend it — `Ingredient`, `Equipment`, and `Collectible` — and each of those is extended in turn by a named concrete subclass used in the world package: `OatMilk` and `EspressoShot` extend `Ingredient`, `MokaPot` extends `Equipment`, and `RareBean`

extends `Collectible`. All eight classes are stored uniformly in one generic container, `Inventory<T extends Item>`.

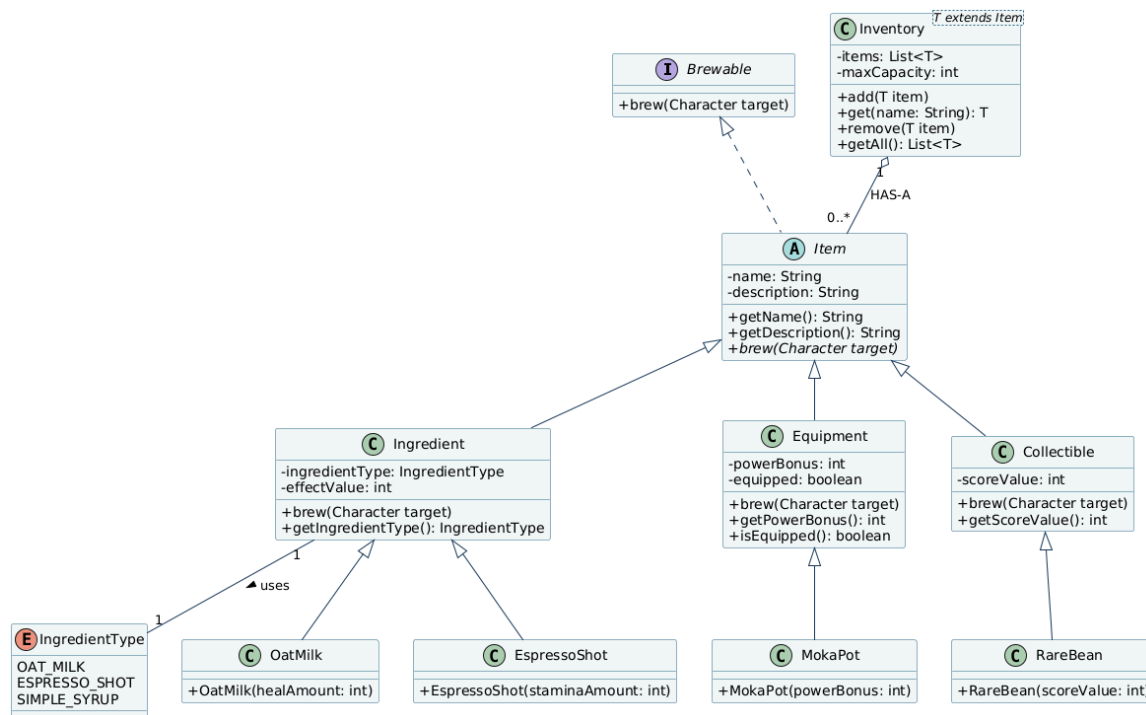


Figure 3: Class diagram of the items package

Figure 4 shows the world package. `Region` and `Encounter` are two separate hierarchies connected by composition: each `Region` holds a `List<Encounter>`, built by `buildEncounterPool()` at game start. `WorldManager` sits on top, holding the three regions in order. It resolves almost every encounter the same way — calling `enc.resolve(barista, engine)` — regardless of whether it's a `BattleEncounter`, `ShopEncounter`, or `RandomEvent`. The one exception is a small `instanceof BattleEncounter` check used purely for cosmetic purposes in the final region, to number Dieng Highlands's three battles ("Battle 1 of 3", "Battle 2 of 3"...) in the console output — the actual resolution logic itself stays fully polymorphic.

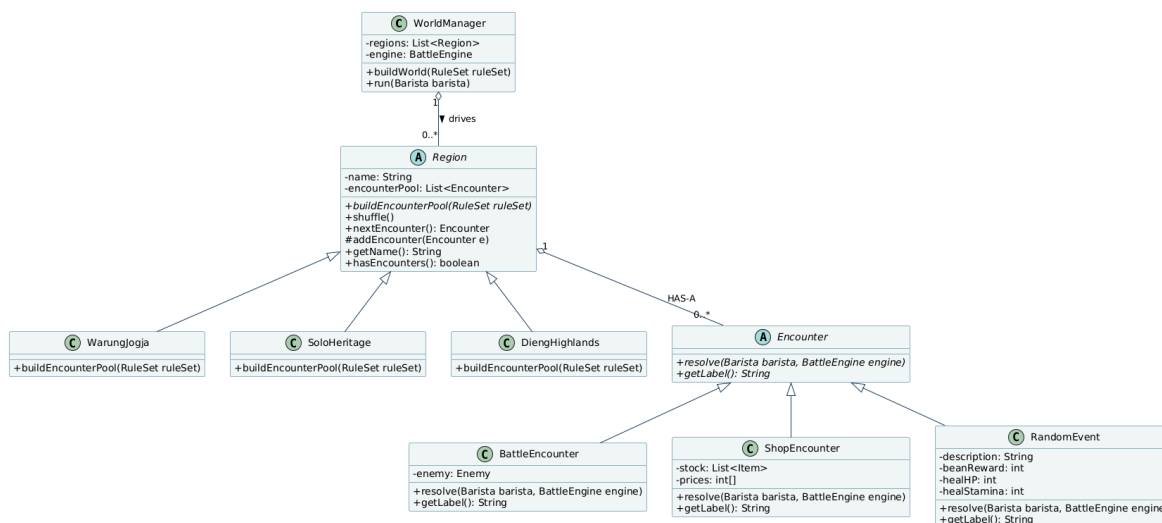


Figure 4: Class diagram of the world package

6. Game Logic and World Extensions

This section documents the gameplay systems that give Java Coffee its simulation depth despite being a console-only project with no graphical interface. With no visuals to lean on, these systems had to carry the weight of making battles feel eventful — timed effects, scaling difficulty, procedural encounters, a boss fight with real stakes — using nothing but console output and the class design from Section 3.

6.1 Status Effects

I implemented three status effects, each extending the abstract `StatusEffect` class with a deliberately different `tick()` implementation

- **Burned** — deals a fixed amount of damage per turn for N turns. Applied by the Brewmaster in Phase 2.

```
Burned.java X
package model;

import exceptions.BattleOverException;

public class Burned extends StatusEffect {
    private int damagePerTurn;

    public Burned(int damagePerTurn, int turns) {
        super("Burned", turns);
        this.damagePerTurn = damagePerTurn;
    }

    @Override
    public void tick(Character target) {
        System.out.println(" " + target.getName() + " is Burned — takes " + damagePerTurn + " damage!");
        try { target.takeDamage(damagePerTurn); }
        catch (BattleOverException e) { System.out.println(" " + e.getMessage()); }
    }
}
```

// model/Brewmaster.java:

```
Brewmaster.java X
} else {
    switch (random.nextInt(2)) {
        case 0:
            attack(target, "explodes with burning rage!", getAttackPower() + 5);
            target.addStatusEffect(new Burned(3, 2));
            break;
        case 1:
            attack(target, "releases a giant steam burst!", getAttackPower());
            target.addStatusEffect(new Steamed(2));
            target.addStatusEffect(new Jittery(1));
            break;
    }
}
```

- **Steamed** — deals 3 damage AND drains 1 stamina per turn for 2 turns. Applied by the Cappuccino card.

```

Steamed.java ×
package model;

import exceptions.BattleOverException;

public class Steamed extends StatusEffect {
    public Steamed(int turns) { super("Steamed", turns); }

    @Override
    public void tick(Character target) {
        System.out.println(" " + target.getName() + " is Steamed — takes 3 damage and loses 1 stamina!");
        try { target.takeDamage(3); }
        catch (BattleOverException e) { System.out.println(" " + e.getMessage()); }
        try { target.useStamina(1); }
        catch (exceptions.InvalidBrewException e) { /* already at 0 */ }
    }
}

```

// Main.java:

```

Main.java ×
deck.addCard(new DebuffCard("Cappuccino", 2, new model.Steamed(2)));

```

- **Jittery** — gives a 50% chance each turn that the affected entity fumbles and loses their turn. Applied by the Angry Customer.

```

Jittery.java ×
package model;

import java.util.Random;

public class Jittery extends StatusEffect {
    private boolean fumbled;

    public Jittery(int turns) { super("Jittery", turns); }

    @Override
    public void tick(Character target) {
        fumbled = new Random().nextDouble() < 0.50;
    }

    public boolean didFumble() { return fumbled; }
    public boolean isJustApplied() { return super.isJustApplied(); }
}

```

// model/AngryCustomer.java:

```
AngryCustomer.java X
@Override
protected void decideAction(Character target) throws BattleOverException {
    switch (random.nextInt(3)) {
        case 0:
            System.out.println(" Angry Customer slams the table!");
            System.out.println(" -> " + target.getName() + " takes " + getAttackPower() + " damage!");
            target.takeDamage(getAttackPower());
            target.addStatusEffect(new Jittery(2));
            break;
        case 1:
            System.out.println(" Angry Customer demands a refund!");
            System.out.println(" -> " + target.getName() + " takes " + getAttackPower() + " damage!");
            target.takeDamage(getAttackPower());
            target.addStatusEffect(new Jittery(2));
            break;
        case 2:
            System.out.println(" Angry Customer spills hot coffee at you!");
            System.out.println(" -> " + target.getName() + " takes " + getAttackPower() + " damage!");
            target.takeDamage(getAttackPower());
            target.addStatusEffect(new Jittery(2));
            break;
    }
}
```

// battle/BattleEngine.java:

```
BattleEngine.java X
// Check if enemy is Jittery and fumbles
boolean enemyFumbled = false;
for (model.StatusEffect se : enemy.getStatusEffects()) {
    if (se instanceof model.Jittery && !se.isJustApplied()
        && ((model.Jittery) se).didFumble()) {
        enemyFumbled = true;
        break;
    }
}
```

Listing 22: Three StatusEffect subtypes — different tick() for each

One important detail I had to handle carefully was the first-turn problem: if a status effect ticked on the same turn it was applied, it would expire one turn too early. I fixed this by adding a `justApplied` flag to the base `StatusEffect` class — `tickStatusEffects()` in `Character` skips the tick and decrement on the first turn, so every effect lasts exactly as long as intended.

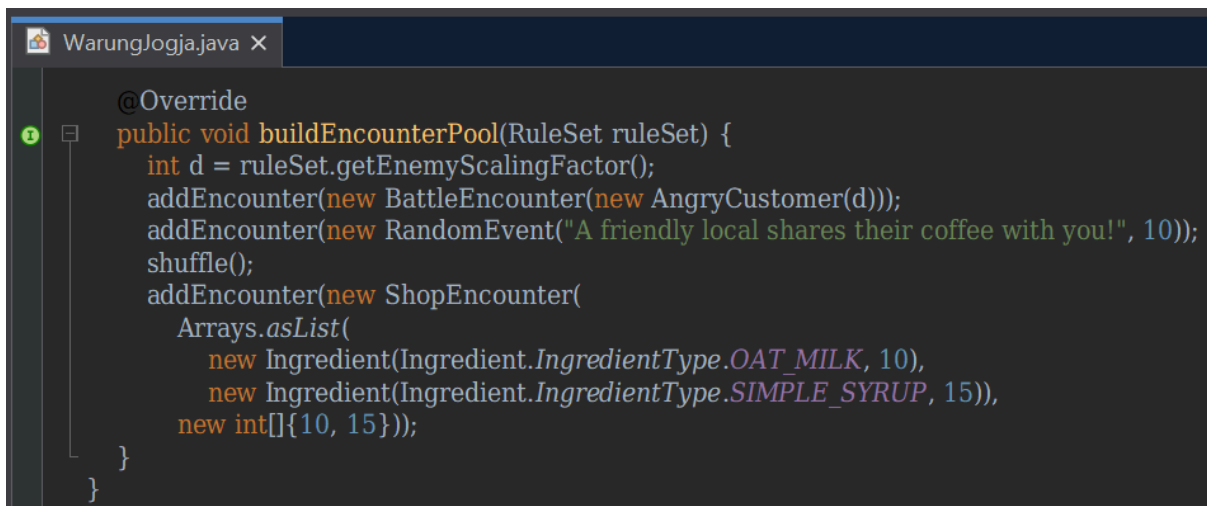
Integration with BattleEngine: Before each turn, `BattleEngine` checks if any `Jittery` effect on the combatant has fumbled, and skips their turn if so. This is the only place in `BattleEngine` that inspects a specific status effect type — all other status logic runs through the polymorphic `tickStatusEffects()` loop automatically.

6.2 Procedural Encounters

Each `Region` builds its own encounter pool in `buildEncounterPool(RuleSet)`. I designed the shuffling behaviour differently for each region:

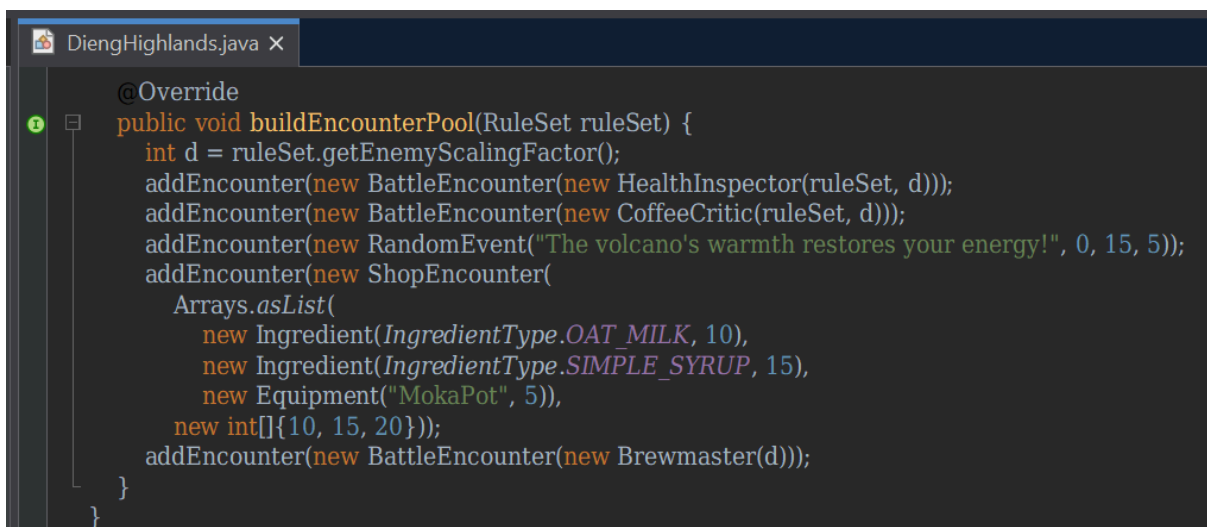
- Warung Jogja and Solo Heritage — the battle and random event are shuffled, so the player never knows which comes first. The shop is added *after* the shuffle, so it's always last, guaranteeing a chance to spend beans before moving on.
- Dieng Highlands — `shuffle()` is never called at all. Encounters run in a fixed order: HealthInspector, CoffeeCritic, a random event, the shop, then the Brewmaster. As the final region, the Brewmaster has to come last no matter what — leaving that to chance could put the final boss before the player's even fully equipped.

// `world/WarungJogja.java`:

A screenshot of a code editor window titled 'WarungJogja.java'. The code is in Java and shows an override of the 'buildEncounterPool' method. It starts with getting the enemy scaling factor 'd', then adds a BattleEncounter with an AngryCustomer, a RandomEvent with a specific message, calls 'shuffle()', and finally adds a ShopEncounter with a list of ingredients and equipment.

```
@Override
public void buildEncounterPool(RuleSet ruleSet) {
    int d = ruleSet.getEnemyScalingFactor();
    addEncounter(new BattleEncounter(new AngryCustomer(d)));
    addEncounter(new RandomEvent("A friendly local shares their coffee with you!", 10));
    shuffle();
    addEncounter(new ShopEncounter(
        Arrays.asList(
            new Ingredient(Ingredient.IngredientType.OAT_MILK, 10),
            new Ingredient(Ingredient.IngredientType.SIMPLE_SYRUP, 15)),
        new int[]{10, 15}));
}
```

// `world/DiengHighlands.java`:

A screenshot of a code editor window titled 'DiengHighlands.java'. The code is in Java and shows an override of the 'buildEncounterPool' method. It follows a fixed order: getting the enemy scaling factor 'd', adding BattleEncounters for HealthInspector and CoffeeCritic, a RandomEvent, a ShopEncounter with ingredients and equipment, and finally a Brewmaster BattleEncounter.

```
@Override
public void buildEncounterPool(RuleSet ruleSet) {
    int d = ruleSet.getEnemyScalingFactor();
    addEncounter(new BattleEncounter(new HealthInspector(ruleSet, d)));
    addEncounter(new BattleEncounter(new CoffeeCritic(ruleSet, d)));
    addEncounter(new RandomEvent("The volcano's warmth restores your energy!", 0, 15, 5));
    addEncounter(new ShopEncounter(
        Arrays.asList(
            new Ingredient(IngredientType.OAT_MILK, 10),
            new Ingredient(IngredientType.SIMPLE_SYRUP, 15),
            new Equipment("MokaPot", 5)),
        new int[]{10, 15, 20}));
    addEncounter(new BattleEncounter(new Brewmaster(d)));
}
```

Listing 23: *WarungJogja (shuffled) vs DiengHighlands (fixed order)*

Once all encounters are built, `WorldManager.run()` calls `enc.resolve(barista, engine)` on each Encounter without any type-checking. `BattleEncounter`, `ShopEncounter`, and `RandomEvent` all implement the same `resolve()` method — `WorldManager` treats them identically and the correct behaviour fires automatically. This is inclusion polymorphism applied to the overworld, exactly the same pattern as the `GameRule` loop in `BattleEngine`.

6.3 Difficulty Scaling

The difficulty setting (1, 2, or 3) is stored in `RuleSet.enemyScalingFactor` at game setup and passed to every enemy constructor when the encounter pool is built. Each enemy uses a simple formula to scale its HP and attack power based on the difficulty value:

Enemy	Difficulty 1	Difficulty 2	Difficulty 3
Angry Customer	50 HP, 8 ATK	60 HP, 10 ATK	70 HP, 12 ATK
Solo Rival	80 HP, 12 ATK	95 HP, 14 ATK	110 HP, 16 ATK
Health Inspector	60 HP, 5 ATK	75 HP, 7 ATK	90 HP, 9 ATK
Coffee Critic	65 HP, 6 ATK	80 HP, 8 ATK	95 HP, 10 ATK
Brewmaster	100 HP, 10 ATK	130 HP, 13 ATK	160 HP, 16 ATK

Listing 24: Difficulty scaling in enemy constructors

// `model/AngryCustomer.java`:

```
AngryCustomer.java X
public AngryCustomer(int difficulty) {
    super("Angry Customer",
        50 + (difficulty - 1) * 10,
        4,
        8 + (difficulty - 1) * 2,
        10);
}
```

// model/RivalBarista.java:

```
RivalBarista.java X
public RivalBarista(String name, int difficulty) {
    super(name,
          80 + (difficulty - 1) * 15,
          6,
          12 + (difficulty - 1) * 2,
          20);
}
```

// model/HealthInspector.java:

```
HealthInspector.java X
public HealthInspector(RuleSet ruleSet, int difficulty) {
    super("Health Inspector",
          60 + (difficulty - 1) * 15,
          5,
          5 + (difficulty - 1) * 2,
          15);
    this.ruleSet = ruleSet;
}
```

// model/CoffeeCritic.java:

```
CoffeeCritic.java X
public CoffeeCritic(RuleSet ruleSet, int difficulty) {
    super("Coffee Critic",
          65 + (difficulty - 1) * 15,
          5,
          6 + (difficulty - 1) * 2,
          18);
    this.ruleSet = ruleSet;
}
```

// model/Brewmaster.java:

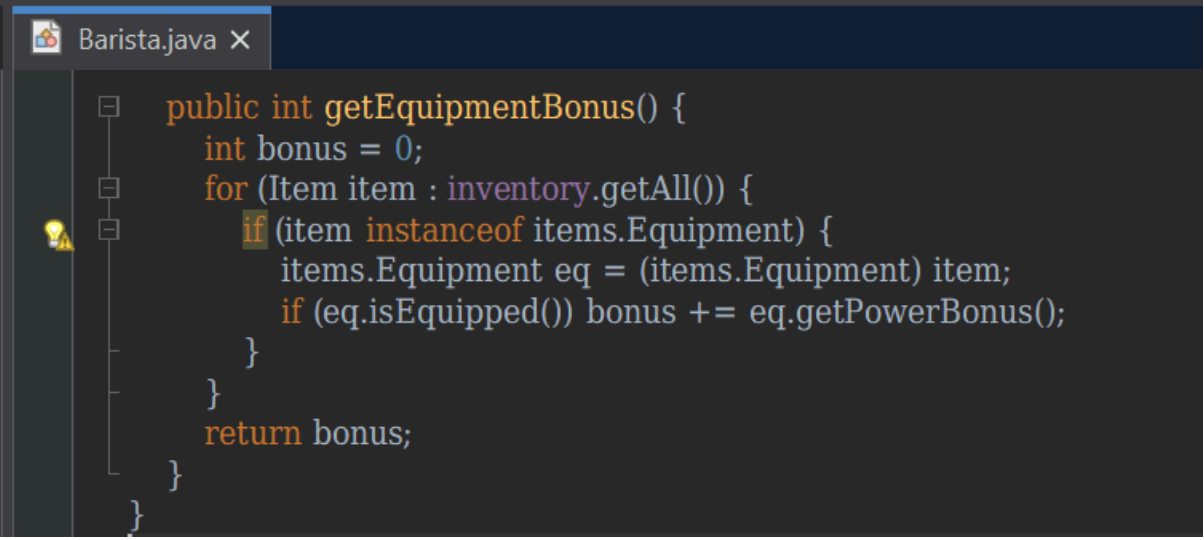
```
Brewmaster.java X
public Brewmaster(int difficulty) {
    super("Brewmaster",
          100 + (difficulty - 1) * 30,
          10,
          10 + (difficulty - 1) * 3,
          100);
}
```

Commentary: I chose to implement scaling as a formula rather than hardcoded values because it means any new enemy I add automatically supports all three difficulty levels by passing the same `difficulty` parameter to `super()`. The scaling is linear — each difficulty level adds a fixed amount of HP and attack — which keeps the game challenging but not exponentially harder.

6.4 Equipment Bonus System

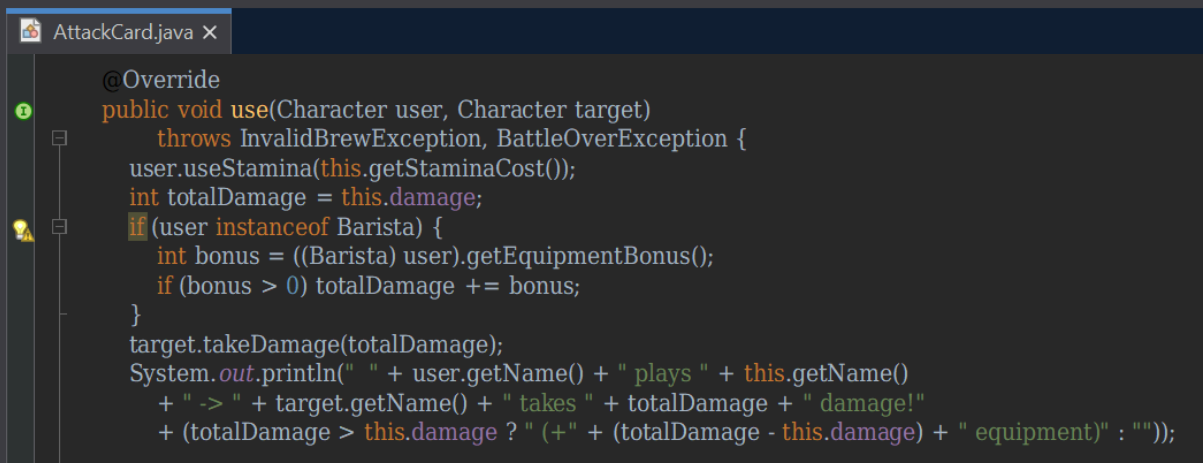
The MokaPot is purchasable from the shop in Solo Heritage and Dieng Highlands for 20 beans. Once bought, it grants a permanent +5 attack bonus for the rest of the run. I implemented this by adding a `getEquipmentBonus()` method to Barista that AttackCard reads every time it is played:

// model/Barista.java:

A screenshot of a code editor showing the `Barista.java` file. The code defines a `getEquipmentBonus()` method that iterates through the `inventory` and checks for equipped equipment items, summing their power bonuses.

```
Barista.java X
public int getEquipmentBonus() {
    int bonus = 0;
    for (Item item : inventory.getAll()) {
        if (item instanceof items.Equipment) {
            items.Equipment eq = (items.Equipment) item;
            if (eq.isEquipped()) bonus += eq.getPowerBonus();
        }
    }
    return bonus;
}
```

//battle/AttackCard.java:

A screenshot of a code editor showing the `AttackCard.java` file. The code shows the `use()` method overriding the base class, which calls `getEquipmentBonus()` on the user if they are a Barista and applies the bonus to the total damage.

```
AttackCard.java X
@Override
public void use(Character user, Character target)
    throws InvalidBrewException, BattleOverException {
    user.useStamina(this.getStaminaCost());
    int totalDamage = this.damage;
    if (user instanceof Barista) {
        int bonus = ((Barista) user).getEquipmentBonus();
        if (bonus > 0) totalDamage += bonus;
    }
    target.takeDamage(totalDamage);
    System.out.println(" " + user.getName() + " plays " + this.getName()
        + " -> " + target.getName() + " takes " + totalDamage + " damage!"
        + (totalDamage > this.damage ? " (" + (totalDamage - this.damage) + " equipment)" : ""));
}
```

Listing 25: Equipment bonus applied in AttackCard.use()

Commentary: Two things worth noting here. First, the bonus is read fresh every time an `AttackCard` is played, not cached — so a second MokaPot bought later stacks automatically, no extra code needed. Second, the `instanceof Barista` check is a deliberate exception to "avoid instanceof": enemies don't carry equipment, so checking here is cleaner than forcing every enemy to implement `getEquipmentBonus()` just to return zero. This is documented as a justified exception in Section 8.4.

6.5 Brewmaster Boss — Two-Phase Battle

The Brewmaster is the final boss and has the most complex logic of all five enemies — a two-phase system where its moves change once HP drops below 50%. The phase check happens right at the top of `decideAction()`, before any move is chosen: if the Brewmaster is still in Phase 1 but its health has just dropped to half or below, `phase` flips to 2, an enrage message prints, and the *same* call to `decideAction()` immediately falls through to the Phase 2 move set — there's no wasted turn where the transition happens but nothing else does.

Phase 1 (above 50% HP) — three possible moves selected randomly:

- Throws hot coffee — base damage
- Slams the counter — base damage + 3
- Blasts hot steam — base damage - 4, applies Steamed

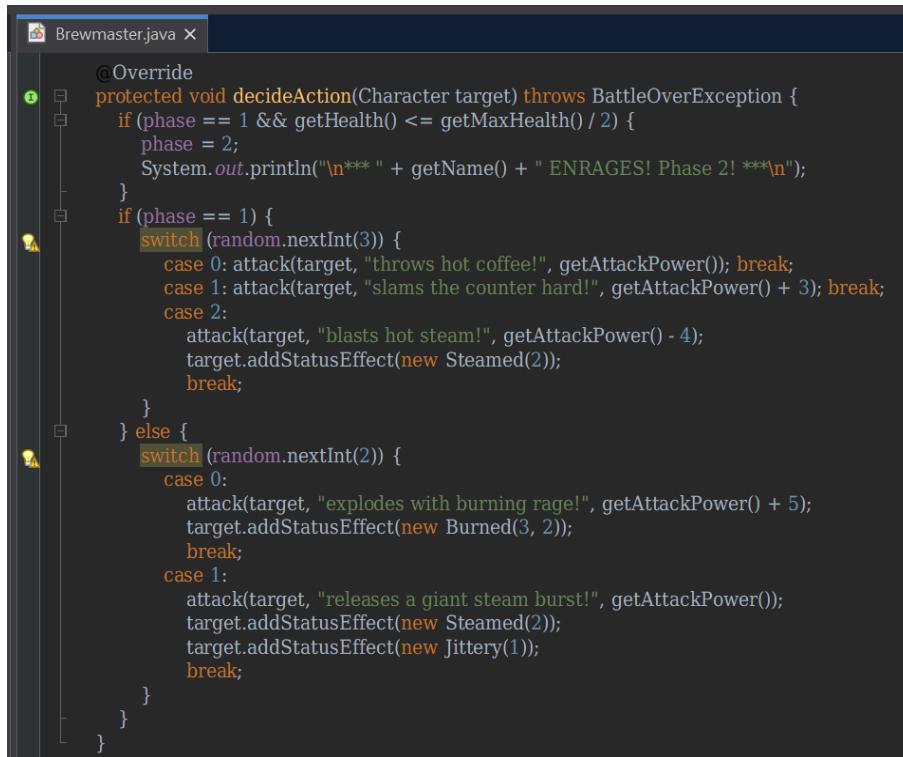
Phase 1 is a mix of a strong hit, a slightly stronger hit, and a weaker hit that trades raw damage for a lingering debuff — giving the player a taste of status effects before the fight gets serious.

Phase 2 (below 50% HP) — two more powerful moves:

- Explodes with burning rage — base damage + 5, applies Burned for 2 turns
- Releases a giant steam burst — base damage, applies both Steamed and Jittery simultaneously

Phase 2 drops the "safe" plain-damage option entirely — every remaining move comes with a status effect attached, and the giant steam burst is the only place in the whole game where two status effects land on the same target from a single move. That stacking is deliberate: it's meant to be the fight's real difficulty spike, arriving exactly when the player has the least room for error.

//model/Brewmaster.java:

A screenshot of a code editor window titled 'Brewmaster.java'. The code is in Java and implements the 'decideAction' method. It starts with an '@Override' annotation and a 'protected void' declaration. The method takes a 'Character target' and throws a 'BattleOverException'. The logic is as follows: if the current phase is 1 and the character's health is less than or equal to half of their maximum health, the phase is set to 2 and a message is printed: '\n*** ' + getName() + ' ENRAGES! Phase 2! ***\n'. If the phase is 1, a random integer between 0 and 2 is generated. A switch statement handles these cases: case 0: attack with 'throws hot coffee!', no change to attack power; case 1: attack with 'slams the counter hard!', increase attack power by 3; case 2: attack with 'blasts hot steam!', decrease attack power by 4, and add a 'Steamed(2)' status effect. If the phase is 2, a random integer between 0 and 1 is generated. A switch statement handles these cases: case 0: attack with 'explodes with burning rage!', increase attack power by 5, and add a 'Burned(3, 2)' status effect; case 1: attack with 'releases a giant steam burst!', no change to attack power, and add both 'Steamed(2)' and 'Jittery(1)' status effects. The code ends with a closing brace for the method.

Listing 26: Brewmaster two-phase battle logic with random move selection

Commentary: The phase check happens at the start of every `decideAction()` call — once HP drops below half, `phase` flips to 2 and the enrage message prints. Because the check runs before the move is picked, the transition and the first Phase 2 move happen on the same turn, so there's no "dead" turn where the enrage message prints but nothing else changes. Phase 2 is noticeably more dangerous since it can stack two status effects, `Steamed` and `Jittery`, in one turn — the giant steam burst is the only move in the whole game where a single action applies two effects at once, which is what makes the last stretch of the fight feel like a real difficulty spike rather than just "the same fight with a bigger number."

6.6 RunSummary and Score Log

I implemented a `RunSummary` object that accumulates statistics silently throughout the entire run and prints a structured report at the end.

`BattleEngine` calls the relevant method at each key moment:

- `summary.recordTurn()` — once per turn, increments the turn counter
- `summary.recordCardPlay()` — every time the player plays a card
- `summary.addDamage(amount)` — every time an attack card deals damage
- `summary.addHeal(amount)` — every time a heal card restores HP

- `summary.recordKill()` — every time an enemy is defeated
- `summary.addScore(points)` — every time an enemy is defeated, adding that enemy's reward value to the running score
- `summary.log(message)` — every battle's outcome, with the turn number attached automatically

One thing that happens outside `BattleEngine` entirely: `summary.setBeans()` is called from `Main`, once, right at the end of the run, after the player has left the world loop — it's the only summary field that isn't updated turn-by-turn during battle, since beans are collected across shops and events, not battles specifically.

At the end of the run, `summary.printReport()` prints the full structured output, combining the running totals with the battle log built up by every `log()` call along the way.

```

-----
                        RUN SUMMARY
-----
Score:                  163
Turns played:           19
Cards played:           12
Enemies defeated:       5
Damage dealt:           112
Healing done:           20
Beans collected:        133
-----

                        BATTLE LOG
-----

[Turn 3] Nasywa defeated Angry Customer.
[Turn 9] Nasywa defeated Solo Rival.
[Turn 11] Nasywa defeated Health Inspector.
[Turn 15] Nasywa defeated Coffee Critic.
[Turn 19] Nasywa defeated Brewmaster.
-----

BUILD SUCCESSFUL (total time: 3 minutes 5 seconds)

```

Listing 27: RunSummary.java — statistics accumulated throughout the run

Commentary: I deliberately kept `RunSummary` separate from `BattleEngine` rather than printing statistics inline during the battle. This separation means the output during battle stays focused on what is happening right now, while the full statistics appear only once at the end. The battle log is particularly useful for the oral exam — it shows exactly which turn each enemy was defeated, making it easy to trace the game flow after a run.

7. Sample Run Transcript

Since Java Coffee is console-only, this section provides an annotated transcript of an actual play session. Each screenshot is taken directly from the running game. Annotations connect each moment to the corresponding code.

7.1 Setup — Configuring Rules and Authoring a GameRule

```
--- Game Setup ---
(Press Enter to use default values)
Deck size   (5-12,   default 7):
Hand size   (2-10,   default 4):
Max stamina (5-20,   default 10):
Difficulty  (1-3,    default 1):
Permadeath  (y/n,    default n):

--- Custom Rules ---
Write your own rules. They fire automatically every turn.
Separate multiple rules with ;

  IF player.health < 30 THEN player.heal(15)
  IF player.health < 30 THEN player.gain(X)
  IF player.stamina < 3 THEN draw(2)
  IF turn % 3 == 0 THEN draw(1)
  COMBO 2 THEN damage_boost_20

X = OatMilk (+10 HP) EspressoShot (+10 ST) SimpleSyrup (+5 HP +5 ST)

Tip: IF player.health < 30 THEN player.gain("OatMilk"); IF turn % 3 == 0 THEN draw(1)

(Press Enter to skip)
Your rules: IF player.health < 30 THEN player.gain("OatMilk"); IF turn % 3 == 0 THEN draw(1)
OK: IF player.health < 30 THEN player.gain("OatMilk")
OK: Periodic Draw: Every 3 turns, gain 1 card(s)
2 rule(s) active this run.

Enter your barista's name: Nasywa

--- Choose Your Barista ---
[1] Standard Barista
[2] Ice Barista      (+2 max stamina)
[3] Espresso Barista (+10 max HP)
Choice (1-3): 1
```

Listing 28: Setup — two rules authored at runtime from player input

Annotation: `RuleParser.parseRules()` splits the two rules on `;`. The first matches the `HEALTH_GAIN` pattern and becomes an `ItemGainRule(30, "OatMilk")`; the second matches `TURN_DRAW` and becomes a `PeriodicDrawRule(3, 1)`. Both are added to `RuleSet` before the first battle begins. Neither object existed before the player typed that line.

7.2 Battle — A Player-Authored Rule Fires

```
--- TURN 3 ---
Nasywa [HP:84/100 ST:6/10] | Angry Customer [HP:20/50 ST:4/4]
[DRAW] RULE BONUS: 1 extra card(s) drawn!

Nasywa's turn (Stamina: 6/10):
Hand (4 cards):
  [1] [ATTACK] Cafe Creme (cost: 1 stamina, 8 dmg)
  [2] [ATTACK] Americano (cost: 3 stamina, 22 dmg)
  [3] [HEAL] Oat Latte (cost: 1 stamina, +20 HP)
  [4] [HEAL] Warm Milk (cost: 0 stamina, +4 ST)
  [0] Pass turn (skip)
Choose card (1-4) or 0 to pass:
> 2
Angry Customer has been defeated!
```

Listing 29: Turn 3 — PeriodicDrawRule fires, giving an extra card before the prompt

Annotation: `PeriodicDrawRule` fires for the first time on Turn 3, printing "[DRAW] RULE BONUS: 1 extra card(s) drawn!" before the hand is shown. It's the same object created back in setup — `BattleEngine` just checks `appliesTo()` and runs `execute()`, without knowing what kind of rule it is. The extra card gave Nasywa Americano (22 dmg), enough to finish off the Angry Customer.

7.3 Battle — The Player Silences an Enemy (BanCard)

```
--- TURN 3 ---
Nasywa [HP:84/100 ST:6/10] | Angry Customer [HP:6/50 ST:4/4]

Nasywa's turn (Stamina: 6/10):
Hand (4 cards):
  [1] [HEAL] Honey Latte (cost: 3 stamina, +22 HP)
  [2] [HEAL] Steamed Milk (cost: 2 stamina, +10 HP)
  [3] [ATTACK] Cafe Creme (cost: 1 stamina, 8 dmg)
  [4] [DEBUFF] Ristretto Lock (cost: 3 stamina, silences enemy 1 turn)
  [0] Pass turn (skip)
Choose card (1-4) or 0 to pass:
> 4
  Nasywa plays Ristretto Lock -> Angry Customer is SILENCED for 1 turn(s)!

Angry Customer's turn:
  Angry Customer is silenced and cannot act!
  Nasywa is no longer Jittery.

Press Enter to continue...
```

Listing 30: Ristretto Lock silences enemy; ATTACK ban and Steamed status in play

→ This transcript is from a separate run with deck size set to 12, which draws all five extra cards including Ristretto Lock and Honey Latte.

Annotation: Ristretto Lock is a `BanCard`, built as `new BanCard("Ristretto Lock", 3, 1)`. Unlike the enemy-side bans seen with Health Inspector and Coffee Critic (which ban a *card type* via `RuleSet.banCardType()`), `BanCard.use()` calls `((Enemy) target).silence(turns)` directly on the target — silencing the whole enemy for `turns` (here, 1 turn), preventing it from acting at all rather than blocking specific cards. This is the player-side mirror of the enemy's ban mechanic, giving both sides symmetric access to the same kind of tempo-control tool.

8. Technical Challenges and Design Decisions

This section covers the design decisions that involved real trade-offs, or where the "textbook" solution turned out to be impractical.

8.1 Single Static Scanner — Preventing Input Buffering Bugs

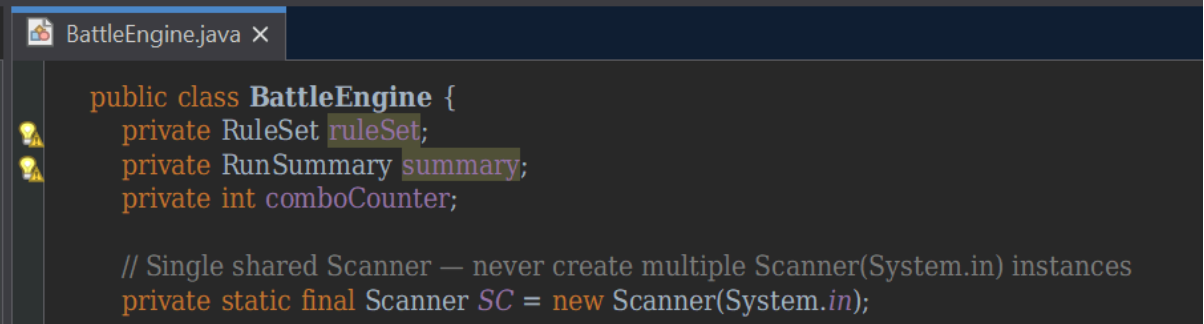
What problem I encountered: Early on, the game would randomly skip a prompt or read the wrong input for the wrong method.

What my first approach was: A new `Scanner(System.in)` inside every method that needed input — `playerTurn()`, the shop, the setup wizard.

Why it didn't work: Multiple `Scanner` instances share the same underlying buffer on `System.in`, so they end up competing for the same bytes — one steals input another was waiting for.

What I did instead: I declared a `private static final Scanner SC = new Scanner(System.in)` at the top of each class that needs input — constructed exactly once, never passed between classes. `BattleEngine`, `ShopEncounter`, and `Main` each have their own static instance.

`//battle/BattleEngine.java:`



```
public class BattleEngine {
    private RuleSet ruleSet;
    private RunSummary summary;
    private int comboCounter;

    // Single shared Scanner — never create multiple Scanner(System.in) instances
    private static final Scanner SC = new Scanner(System.in);
}
```

Listing 31: BattleEngine.java — single static Scanner declared once per class

Why it matters for OOP: Looking back, this is encapsulation applied to a resource. The `Scanner` is private to the class that owns it — no other class can touch it. Having multiple `Scanner` instances sharing the same stream was essentially shared mutable state, which is exactly what encapsulation is supposed to prevent.

8.2 While Loop Instead of Recursion for Re-prompt

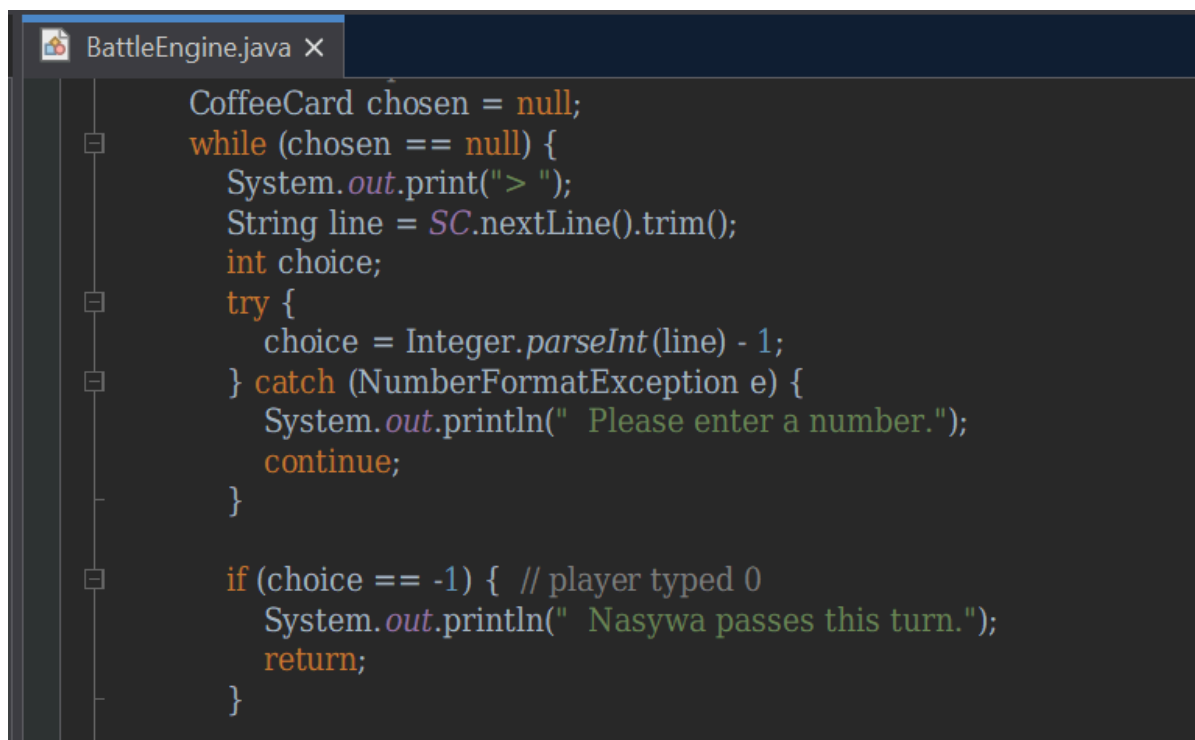
What problem I encountered: During testing I realised that if a player kept entering invalid inputs for a long time, the game could theoretically crash. It wasn't something that happened often, but it was a real risk.

What my first approach was: I wrote `playerTurn()` to call itself recursively whenever the player entered an invalid card number, chose a banned card, or picked a card they couldn't afford. It felt natural — "if the input is wrong, just ask again."

Why it didn't work: Recursion creates a new stack frame every time. A player who makes 50 invalid inputs in a row would cause a `StackOverflowError`. In a card game where the player might genuinely be stuck without playable cards for several turns, this is a realistic scenario, not just a theoretical one.

What I did instead: I replaced all re-prompt recursion with a `while (chosen == null)` loop. The loop keeps prompting until the player makes a valid, affordable, non-banned selection, then exits cleanly.

//battle/BattleEngine.java:



```
CoffeeCard chosen = null;
while (chosen == null) {
    System.out.print("> ");
    String line = SC.nextLine().trim();
    int choice;
    try {
        choice = Integer.parseInt(line) - 1;
    } catch (NumberFormatException e) {
        System.out.println(" Please enter a number.");
        continue;
    }

    if (choice == -1) { // player typed 0
        System.out.println(" Nasywa passes this turn.");
        return;
    }
}
```

Listing 32: BattleEngine.java — while loop replaces recursive re-prompt

Why it matters for OOP: This taught me that elegant-looking recursive designs can have hidden costs. The loop solution is slightly less elegant to read but significantly safer in practice. Good OOP isn't just about clean structure — it's also about making the system robust under real usage conditions.

8.3 Defensive Copy in the Rule Evaluation Loop

What problem I encountered: When I was designing the rule evaluation loop, I started thinking about what would happen if a rule's `execute()` method added a new rule to the `RuleSet` while the loop was still iterating. Java would throw a `ConcurrentModificationException`.

What my first approach was: My first instinct was to use a defensive copy — iterate over `new ArrayList<>(rules)` instead of the original list. This is the standard Java solution to this problem.

Why it didn't work: After analysing the actual implementation more carefully, I realised the defensive copy wasn't needed — at least not right now. The player's rules are added before the first battle and never modified during battle. Enemy card bans call `banCardType()`, which modifies a separate `Set<CardType>` — not the `List<GameRule>`. So the rule list is never actually modified during iteration.

`//battle/BattleEngine.java:`

```
BattleEngine.java X
private void applyCustomRules(BattleContext context) {
    List<GameRule> rules = ruleSet.getRules();
    for (GameRule rule : rules) {
        if (rule.appliesTo(context)) {
            rule.execute(context);
        }
    }
}
```

Listing 33: BattleEngine.java — applyCustomRules() loop safe without defensive copy

Why it matters for OOP: This is a good example of not over-engineering. Adding a defensive copy "just in case" would have added complexity without solving a real problem. Understanding why the current design is safe — and documenting it — is more valuable than blindly applying a pattern.

8.4 instanceof in BanCard and AttackCard

What problem I encountered: Two places in my code needed to check the concrete type of an object before calling a subtype-specific method — `BanCard` needs to know if the target is an `Enemy` before silencing it, and `AttackCard` needs to know if the user is a `Barista` before applying the equipment bonus.

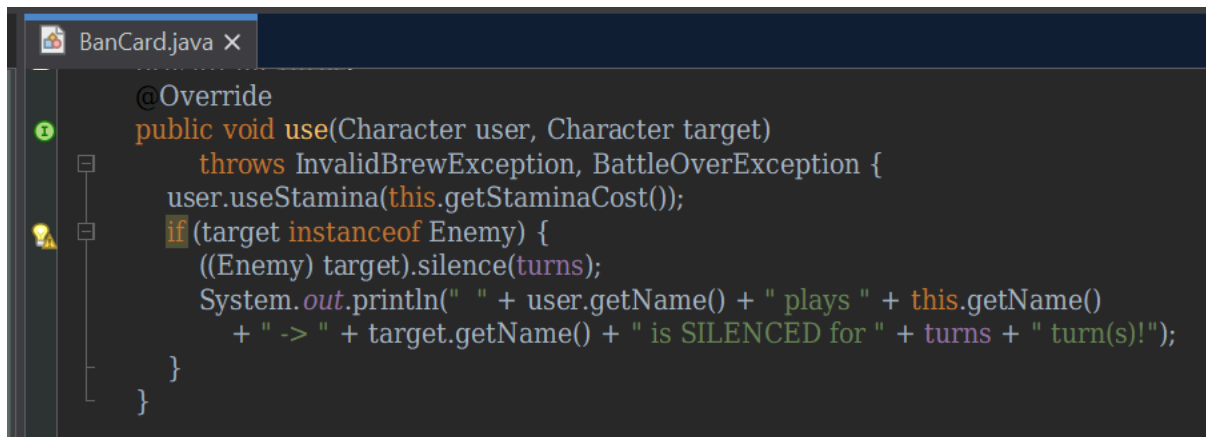
What my first approach was: My first instinct was to add `silence()` to the abstract `Character` class, and `getEquipmentBonus()` to `Character` as well — that way I could call them without any type checking. This is the "pure OOP" approach.

Why it didn't work: Adding `silence()` to `Character` would force `Barista`, `SpecialistBarista`, and every other `Character` subtype to implement a method that does nothing

for them. Same for `getEquipmentBonus()` on enemies. This is a leaky abstraction — the Character contract would be polluted with concerns that only apply to one specific subtype.

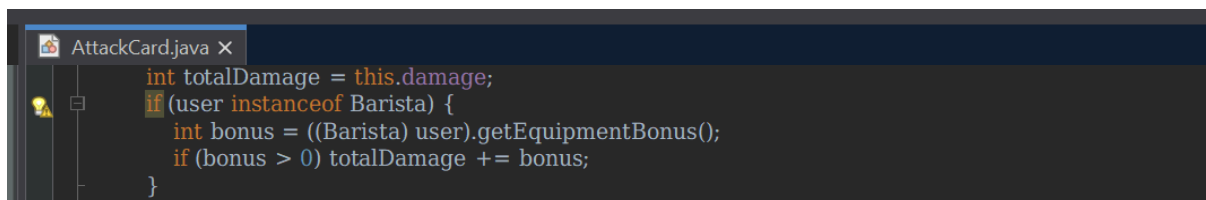
What I did instead: I kept the `instanceof` checks, confined to the `use()` methods of BanCard and AttackCard. Both checks are guarded — the code only proceeds if the cast is safe. I documented both as deliberate exceptions to the "avoid instanceof" principle.

//battle/BanCard.java:



```
@Override
public void use(Character user, Character target)
    throws InvalidBrewException, BattleOverException {
    user.useStamina(this.getStaminaCost());
    if (target instanceof Enemy) {
        ((Enemy) target).silence(turns);
        System.out.println(" " + user.getName() + " plays " + this.getName()
            + " -> " + target.getName() + " is SILENCED for " + turns + " turn(s)!");
    }
}
```

//battle/AttackCard.java:



```
int totalDamage = this.damage;
if (user instanceof Barista) {
    int bonus = ((Barista) user).getEquipmentBonus();
    if (bonus > 0) totalDamage += bonus;
}
```

Listing 34: BanCard.java and AttackCard.java — deliberate instanceof checks

Why it matters for OOP: This taught me that OOP principles are guidelines, not absolute rules. Sometimes a small, well-documented `instanceof` is genuinely better than a leaky abstraction. The key is to understand why you're breaking the rule and document it clearly.

8.5 Hand Management — Clearing Between Battles

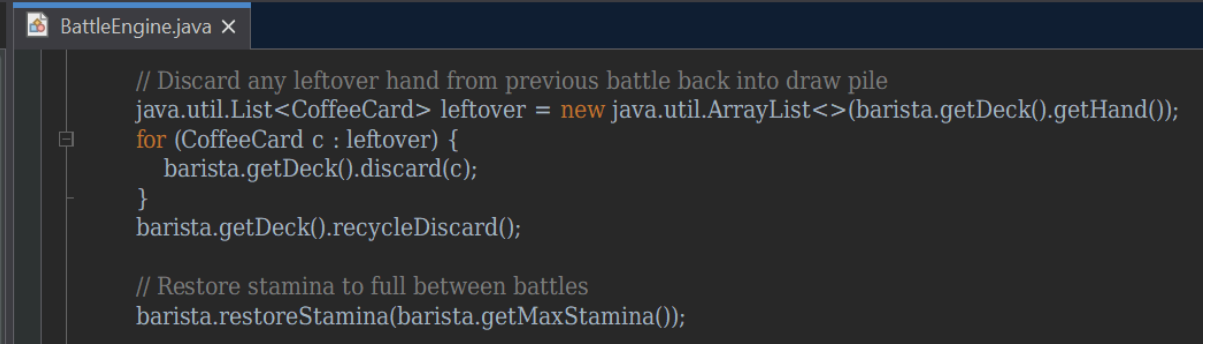
What problem I encountered: After my first complete playthrough of the game, I noticed the second battle started with extra cards in hand — cards left over from the first battle, plus the new draws. This caused duplicates and a hand size larger than configured.

What my first approach was: My first thought was to just clear the hand list directly at the start of each battle — `hand.clear()` or something similar.

Why it didn't work: The hand list is private to CoffeeDeck — external code cannot access it directly. That's encapsulation working correctly. But it meant I had to find a different way to clean up the hand between battles.

What I did instead: I copy the hand to a new list before iterating — without the copy, calling `discard()` on each card would modify the hand list while iterating over it, throwing a `ConcurrentModificationException`. Then after all cards are discarded, I call `recycleDiscard()` to shuffle everything back into the draw pile, and `restoreStamina()` to reset stamina to full.

//battle/BattleEngine.java:



```

// Discard any leftover hand from previous battle back into draw pile
java.util.List<CoffeeCard> leftover = new java.util.ArrayList<>(barista.getDeck().getHand());
for (CoffeeCard c : leftover) {
    barista.getDeck().discard(c);
}
barista.getDeck().recycleDiscard();

// Restore stamina to full between battles
barista.restoreStamina(barista.getMaxStamina());

```

Listing 35: BattleEngine.java — hand cleanup using defensive copy before discarding

Why it matters for OOP: This is encapsulation working exactly as intended. The hand, draw pile, and discard pile are private to `CoffeeDeck` — I couldn't just clear them from outside. The fix had to go through `CoffeeDeck`'s public interface, which kept the invariant intact. Encapsulation forced me to find the right solution.

8.6 Unifying Player Rules and Enemy Card Bans

What problem I encountered: When I implemented `HealthInspector`'s ability to ban ATTACK cards, I needed a way to track which card types were banned and for how long. The game also needed to show [BANNED] in the hand display and prevent banned cards from being selected.

What my first approach was: My first draft used simple boolean flags — `attackCardsBanned = true`, `healCardsBanned = true`. It was quick to write and easy to understand.

Why it didn't work: Every time I wanted to add a new enemy that banned a different card type, I would need to add a new boolean field to `RuleSet`. If I wanted to ban DEBUFF or COMBO cards, I'd need `debuffCardsBanned` and `comboCardsBanned`. That's not extensible — it violates the Open/Closed Principle.

What I did instead: I replaced the boolean flags with a `Map<CardType, Integer> bannedCardDurations`, giving each banned type its own independent countdown. A single `isCardAllowed(CardType)` method handles all ban types — called by both the hand display and the card selection loop, so they can never go out of sync.

//battle/RuleSet.java:



```
public void banCardType(CardType type, int turns) {
    bannedCardDurations.put(type, turns);
}

public void tick() {
    Iterator<Map.Entry<CardType, Integer>> it = bannedCardDurations.entrySet().iterator();
    while (it.hasNext()) {
        Map.Entry<CardType, Integer> entry = it.next();
        int remaining = entry.getValue() - 1;
        if (remaining <= 0) {
            it.remove();
        } else {
            entry.setValue(remaining);
        }
    }
}

public boolean isCardAllowed(CardType type) {
    return allowedCardTypes.contains(type) && !bannedCardDurations.containsKey(type);
}
```

Listing 36: RuleSet.java — Set-based card ban system with duration countdown

Why it matters for OOP: This is the Open/Closed Principle in practice. The boolean flag approach was closed for extension — every new ban type required modifying existing code. The Map-based approach is open for extension — new ban types work automatically without any changes to RuleSet, and because each type gets its own counter in the map, two bans starting on different turns expire independently instead of sharing one timer.

9. Extensibility and Future Enhancements

I designed every layer of Java Coffee to be open for extension without requiring modification of existing classes. Here is what can be added and exactly what it would take:

- **New card types** — extend CoffeeCard and implement `use()`. BattleEngine, CoffeeDeck, and RuleSet require zero changes. The new card appears in the deck and hand automatically.
- **New enemy types** — extend Enemy and implement `decideAction()`. BattleEngine requires zero changes. The new enemy can be added to any Region's encounter pool immediately.
- **New regions and encounters** — extend Region and Encounter, then add to WorldManager. All other classes are unaffected.
- **New GameRule patterns** — add a new class implementing GameRule, and add one Pattern and one `parseLine()` branch to RuleParser. BattleEngine and RuleSet

require zero changes. The new rule type becomes available to the player at the next game start.

- **New status effects** — extend `StatusEffect` and implement `tick()`. `Character.tickStatusEffects()` handles the new effect automatically through the polymorphic loop — no changes needed anywhere else.

Concrete extensions I would add next:

- A fourth region — **Toraja Coffee Highlands** — as a harder optional final challenge after Dieng Highlands.
- A new `SpecialistBarista` — **Kopi Tubruk** specialization (+15 max HP but reduced stamina regen per turn) — adding a third playstyle focused on surviving longer rather than playing more cards.
- A **PoisonCard** (DEBUFF) that applies a new `DripPoison` status effect — damage that increases by 1 each turn it is active, rewarding early application.
- A new `GameRule` pattern — `IF enemy.health < N THEN damage_boost_N` — that fires when the enemy is close to defeat, giving the player a finishing burst.

Each of these extensions requires only new classes — no modification of `BattleEngine`, `Character`, `CoffeeDeck`, `RuleSet`, or `WorldManager`. That is the Open/Closed Principle demonstrated in practice.

10. Reflection and Learning Outcomes

Building Java Coffee taught me things about OOP that I couldn't have learned just from studying theory.

➤ Information hiding saved me from a bug I hadn't written yet.

When I made `Enemy.act()` final and hid `decideAction()` behind it, I wasn't planning ahead — I did it because I realised `BattleEngine` could accidentally call `decideAction()` directly and trigger a special move twice. The design fixed a bug before it existed. That was the moment information hiding stopped being a textbook concept and became something genuinely useful.

➤ Not all four polymorphisms are equally useful in practice.

Inclusion polymorphism — the `GameRule` loop, enemy dispatch, encounter resolution — drove almost everything. I used it everywhere. Parametric polymorphism saved me from casting errors in `CoffeeDeck<T>` and `Inventory<T>`. Overloading made

`draw()` cleaner. I used as little coercion as possible. I only understood which one to reach for by actually building the system.

➤ **Composition is easier to change than inheritance.**

Early on I had Barista set up with inheritance where composition made more sense. When I refactored it to Barista HAS-A CoffeeDeck, everything became easier to work with. I won't make that mistake again.

➤ **Single-responsibility per package made debugging faster.**

When the Jittery bug appeared — status effects ticking on the wrong turn — I knew immediately it had to be inside the model package. It couldn't come from the world or items because those packages don't touch status effects. That isolation saved me a lot of time.

11. Conclusion

When I started this project, I thought OOP was mostly about writing clean code. By the end of it, I realised it's about making decisions — and living with them.

Java Coffee — The Barista's Quest covers every OOP concept required by the course: abstraction and encapsulation in the Character hierarchy, information hiding in Enemy's final `act()` and protected `decideAction()`, a three-level inheritance hierarchy in SpecialistBarista, subtyping in BattleEngine treating Barista and Enemy uniformly as Character, multi-typing via CoffeeCard implementing both Brewable and Describable, all four kinds of polymorphism, five packages, composition throughout, three parallel hierarchies, reuse at four levels, extensibility at every layer, and five custom exceptions with deliberately different recovery strategies.

The feature I am most proud of is the GameRule system. When Prof. Distefano's feedback said the player must define rules, not select them, I wasn't sure how to build it. The IF/THEN parser, the six concrete rule classes, the fact that enemies can inject their own rules mid-battle using the same mechanism — none of that was in my original plan. It grew out of trying to solve a real design problem, and I think it shows.

BattleEngine has not changed since I first wrote it — despite the rule system growing from two rule types to seven, new enemies being added, and the card system expanding. That's the thing about OOP done right: the system grows without breaking. I didn't fully understand that until I built something big enough to see it happen.

The source code is available at: github.com/nasywaarr/barista-quest.